

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Чувашский государственный университет имени И.Н. Ульянова»
(ФГБОУ ВО «ЧГУ им. И.Н. Ульянова»)

Факультет информатики и вычислительной техники
Кафедра вычислительной техники

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКИЙ ПРОЕКТ)
по направлению подготовки
09.03.01 Информатика и вычислительная техника,
направленность (профиль) «Программное обеспечение средств
вычислительной техники и автоматизированных систем»

на тему Разработка динамической библиотеки на C++ для моделирования клеточных автоматов

Обучающийся

подпись, дата

Ярдыков Э.Е.

ФИО

Руководитель, старший
преподаватель кафедры
вычислительной техники,
ст. пр.

подпись, дата

Юрьева Е.В.

ФИО

Руководитель,
доцент кафедры
вычислительной техники,
к.т.н., доцент

подпись, дата

Ржавин В.В.

ФИО

Заведующий кафедрой
вычислительной техники,
к. пед.н., доцент

подпись, дата

Щипцова А.В.

ФИО

Работа выполнена на базе кафедры вычислительной техники ФГБОУ ВО «ЧГУ им. И.Н. Ульянова»

Чебоксары 2026

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Чувашский государственный университет имени И.Н. Ульянова»
(ФГБОУ ВО «ЧГУ им. И.Н. Ульянова»)

Факультет информатики и вычислительной техники
Кафедра вычислительной техники

ЗАДАНИЕ
на выпускную квалификационную работу
(бакалаврский проект)

Обучающемуся Ярдыкову Эдуарду Евгеньевичу группа ИВТ-41-22

1. Тема работы: Разработка динамической библиотеки на C++ для моделирования клеточных автоматов, утверждена приказом по университету № 1787ст от 16 апреля 2026 г.

2. Срок сдачи ВКР на кафедру 11 июня 2026 г.

3. Исходные данные к работе:

– Intel Intrinsics Guide (<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>),

– Средства разработки: C++, Visual Studio 2026

4. Содержание расчетно-пояснительной записки (перечень подлежащих разработке вопросов)

1 Аналитический раздел

1.1 Библиотеки в программировании

1.2 Характеристика клеточных автоматов и сферы применения

1.3 Формулировка проблемы

1.4 Анализ существующих аналогов

1.5 Актуальность разработки

1.6 Необходимая информация о CPU

2 Проектно-конструкторский раздел

2.1 Выбор средств разработки

2.1.1 Выбор языка программирования

2.1.2 Выбор стандарта языка программирования

2.1.3 Использование SISD и SIMD

2.1.4 Выбор среды разработки

2.1.5 Выбор операционной системы

2.2 Архитектура библиотеки

2.2.1 Способ взаимодействия и работа с памятью

2.2.2 Архитектура

2.3 Алгоритмы и структуры данных

2.3.1 ABI – интерфейс бинарного взаимодействия

2.3.2 Middleware – диспетчер реализаций

2.3.3 Вычисление суммы соседей и применение правил. Аппаратно-зависимые

бэкенды

2.3.4 Реализация алгоритмов вычисления суммы соседей

2.3.5 Реализация алгоритмов применения правил В и S

2.3.6 Плоский массив

2.3.7 Результат выполнения

2.3.8 Класс с клеточным автоматом

2.4 Бенчмарк

3 Экспериментальный раздел

3.1 Среда и методика испытаний

3.2 Измерение производительности для поиска сумм соседей

3.3 Измерение производительности для применения правил В и S

3.4 Сравнение общего времени выполнения

3.5 Руководство оперативного пользователя

Заключение

Список использованных источников

Приложения

5. Тема углубленной проработки: Разработка эффективных алгоритмов суммирования клеток и применения правил В/S в клеточных автоматах для оптимизации C++ библиотеки.

6. Перечень графического материала (слайдов презентации)

- 1) Слайд 1. Название темы;
- 2) Слайд 2. Актуальность;
- 3) Слайд 3. Цель и задачи;
- 4) Слайд 4. Выбор средств разработки;
- 5) Слайд 5. Архитектура;
- 6) Слайд 6. Декомпозиция задачи;
- 7) Слайд 7. SISD и SIMD;
- 8) Слайд 8. Производительность разработанных алгоритмов поиска сумм соседей;
- 9) Слайд 9. Производительность разработанных алгоритмов применения правил В и S;

10) Слайд 10. Производительность общих алгоритмов;

11) Слайд 11. Заключение.

7. Календарный график выполнения работы

- Аналитический раздел к 21.05.2026
- Проектно-конструкторский раздел к 30.05.2026
- Экспериментальный раздел к 07.06.2026
- Проверка текста расчетно-пояснительной записки в системе Антиплагиат

08.06.2026 г.

– Оформление расчетно-пояснительной записки и подготовка презентации к 08.06.2026

- Получение отзыва руководителя ВКР 11.06.2026 г.

8. Рекомендуемая литература

1) Информатика и вычислительная техника: учебн.-метод. пособие для выполнения выпускной квалификационной работы бакалавра / сост. А.В. Щипцова, В.В. Ржавин. 2018. - 60 с.

2) Intel® Intrinsic Guide // Intel URL:
<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html> (дата обращения: 10.05.2026).

3) ГОСТ 7.32— 2017 // МЕЖГОСУДАРСТВЕННЫЙ СОВЕТ ПО СТАНДАРТИЗАЦИИ, МЕТРОЛОГИИ И СЕРТИФИКАЦИИ (МГС) URL:
https://cs.msu.ru/sites/cmc/files/docs/2021-11gost_7.32-2017.pdf (дата обращения: 11.05.2026).

Дата выдачи задания 13 апреля 2026 г.

Руководитель _____ / Юрьева Елена Владимировна
(подпись) (Ф.И.О.)

Руководитель _____ / Ржавин Вячеслав Валентинович
(подпись) (Ф.И.О.)

Задание принял к исполнению «___» апреля 2026 г.

Обучающийся _____ / Ярдыков Эдуард Евгеньевич
(подпись) (Ф.И.О.)

«УТВЕРЖДАЮ»

Заведующий кафедрой _____ / Щипцова Анна Владимировна
(подпись) (Ф.И.О.)

«___» апреля 2026 г.

АННОТАЦИЯ

В данной выпускной работе была разработана высокопроизводительная динамическая библиотека на C++ для моделирования клеточных автоматов.

В ходе выполнения был проведён анализ существующих решений, поиск необходимой информации о процессоре для оптимизации, реализованы и проанализированы различные алгоритмы решения для моделирования. Приведено описание всех этапов создания, тестирования и анализа алгоритмов.

При разработке использовался язык C++ спецификации ISO/IEC 14882:2024 (C++23), среда разработки Visual Studio 2026.

Созданный продукт позволяет использовать библиотеку в научных вычислениях, симуляциях, играх и т.п., что используют клеточные автоматы.

ANNOTATION

In this final-year project involved the development of a high-performance dynamic library in C++ for simulating cellular automata.

During the course of the project, an analysis of existing solutions was carried out, the necessary information regarding the processor was gathered for optimization purposes, and various solution algorithms for simulation were implemented and analyzed. A description of all stages of the creation, testing and analysis of the algorithms is provided.

The project was developed using C++ ISO/IEC 14882:2024 (C++23) and the Visual Studio 2026 development environment.

The resulting product enables the library to be used in scientific computing, simulations, games and similar applications that utilize cellular automata.

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяют следующие термины с соответствующими определениями.

Клеточный автомат (КА) — это дискретная модель, состоящая из регулярной решётки ячеек любой размерности. Каждая из ячеек решётки может находиться в одном из конечного множества состояний. При этом для каждой ячейки определено такое понятие, как «окрестность». В окрестность некой ячейки, например, могут входить все ячейки, находящиеся на расстоянии не более 2 от неё. Существуют правила, определяющие то, как ячейки взаимодействуют друг с другом и переходят в следующее поколение (состояние). Правила эти, в основном, представлены математическими (программируемыми) функциями, которые зависят от текущего состояния ячеек и от состояния их соседей.

Игра «Жизнь» — клеточный автомат, придуманный английским математиком Джоном Конвеем в 1970 году. Это игра без игроков, в которой человек создаёт начальное состояние, а потом лишь наблюдает за её развитием.

Правила B и S — правила «рождения» (Birth) и «выживания» (Survive) клетки в контексте клеточных автоматов.

SISD (Single Instruction, Single Data) — классическая компьютерная архитектура, где единственный процессор выполняет одну инструкцию, которая обрабатывает одну единицу данных за раз.

SIMD (Single Instruction, Multiple Data) — это класс процессорных инструкций, позволяющий выполнить одну операцию одновременно над несколькими элементами данных. Это фундаментальная технология, которая обеспечивает параллелизм вычислений на уровне одного ядра процессора. Вместо того чтобы обрабатывать данные по одному, SIMD позволяет процессору захватывать целый «пакет» данных и применять к нему одну команду за один такт.

SSE (Streaming SIMD Extensions) – расширение инструкций процессора для потоковой обработки в режиме SIMD, т.е. обработка массива данных одной инструкцией. Использует регистры 128 бит.

AVX (Advanced Vector Extensions) – дальнейшее развитие SSE. Использует регистры 256 / 512 бит.

Окрестность Мура – понятие из теории клеточных автоматов, определяющее соседние ячейки на квадратной сетке. Для любой центральной клетки включает 8 соседних клеток, которые имеют с ней общую вершину или сторону (расположены горизонтально, вертикально и по диагонали).

ABI (Application Binary Interface) – набор соглашений для доступа приложения к динамической библиотеке (или иных систем).

CRTP (Curiously Recurring Template Pattern) – идиома в языке C++, в которой класс-наследник использует себя в качестве аргумента шаблона для своего базового класса. Эта идиома позволяет достигнуть полиморфизма во время компиляции, а не во времени выполнения.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	7
ВВЕДЕНИЕ.....	11
1 Аналитический раздел.....	14
1.1 Библиотеки в программировании.....	14
1.2 Характеристика клеточных автоматов и сферы применения.....	15
1.3 Формулировка проблемы	16
1.4 Анализ существующих аналогов.....	17
1.5 Актуальность разработки	18
1.6 Необходимая информация о CPU	19
Выводы по разделу.....	21
2 Проектно-конструкторский раздел	22
2.1 Выбор средств разработки	22
2.1.1 Выбор языка программирования.....	22
2.1.2 Выбор стандарта языка программирования	22
2.1.3 Использование SISD и SIMD	23
2.1.4 Выбор среды разработки	24
2.1.5 Выбор операционной системы	24
2.2 Архитектура библиотеки.....	25
2.2.1 Способ взаимодействия и работа с памятью.....	25
2.2.2 Архитектура.....	27
2.3 Алгоритмы и структуры данных	29
2.3.1 ABI – интерфейс бинарного взаимодействия	29
2.3.2 Middleware – диспетчер реализаций	30
2.3.3 Вычисление суммы соседей и применение правил. Аппаратно- зависимые бэкенды	34
2.3.4 Реализация алгоритмов вычисления суммы соседей	36
2.3.5 Реализация алгоритмов применения правил В и S.....	38
2.3.6 Плоский массив	39

2.3.7Результат выполнения	43
2.3.8Класс с клеточным автоматом	44
2.4 Бенчмарк	47
Выводы по разделу.....	49
3 Экспериментальный раздел	51
3.1 Среда и методика испытаний.....	51
3.2 Измерение производительности для поиска сумм соседей.....	51
3.3 Измерение производительности для применения правил В и S	60
3.4 Сравнение общего времени выполнения.....	66
3.5 Руководство оперативного пользователя	69
Выводы по разделу.....	70
ЗАКЛЮЧЕНИЕ	71
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	72
Приложение А	74
Приложение Б	75
Приложение В.....	82
Приложение Г	86

ВВЕДЕНИЕ

Актуальность работы связана с необходимостью создания удобной и производительной динамической библиотеки для моделирования клеточных автоматов. Динамические библиотеки на C++ позволяют оформить сложную реализацию в виде готовых модулей, которые можно легко подключать к различным проектам. Такой подход позволяет отделить внутреннюю оптимизацию вычислений моделирования клеточного автомата от внешнего кода.

Клеточные автоматы — эффективный способ дискретного моделирования. Они применяются для симуляции физических, химических и общественных процессов, например, таких как: течение жидкостей, переноса тепла, диффузии, распространении огня и динамики социальных систем. Также клеточные автоматы используются в высокопроизводительных вычислительных системах для анализа сложных сред. Дополнительно этот метод находит применение в компьютерных играх, например для процедурной генерации ландшафтов и пещер.

Существующие реализации клеточных автоматов имеют ряд серьезных недостатков, такие как низкая производительность для интерактивных симуляций, жёсткая привязка к конкретной платформе или языку программирования, что мешает их интеграции. Так как рост тактовых частот процессоров замедлился, а дальнейшее повышение производительности возможно только за счёт оптимизации кода, а также использовании SIMD-инструкций. Поэтому разработка динамической библиотеки на C++ для моделирования клеточных автоматов, сочетающей высокую скорость работы и удобный интерфейс, является актуальной задачей.

Проблема исследования заключается в противоречии между потребностью в быстром и универсальном моделировании клеточных автоматов и отсутствием готовых библиотечных решений, которые сочетают

высокую производительность за счёт аппаратных оптимизаций с удобным кроссплатформенным интерфейсом интеграции.

Объектом исследования стала высокопроизводительная динамическая библиотека на C++ для моделирования клеточных автоматов.

Предметом исследования стало проектирование и реализация высокопроизводительных алгоритмов и структур данных для моделирования клеточных автоматов для динамической библиотеки на языке C++.

Цель работы — проектирование архитектуры и программная реализация высокопроизводительной библиотеки на языке C++ для моделирования клеточных автоматов, обеспечивающей эффективное использование аппаратных ресурсов процессора, а также простоту интеграции.

Для достижения цели поставлены следующие задачи:

- разработать многоуровневую архитектуру библиотеки (ABI, middleware и аппаратно-зависимые бэкенды);
- спроектировать специализированные структуры данных для непрерывного хранения полей симуляции, обеспечивающие локальность кэша и корректную поддержку векторных операций;
- реализовать ядро симуляции, алгоритмы подсчета соседей и применения правил (B/S) под различные наборы инструкций аппаратной среды исполнения (Scalar, SSE, AVX);
- обеспечить межъязыковое взаимодействие через стабильный C-совместимый ABI, исключающий копирование данных.

Научная новизна работы заключается в применении CRTP для устранения минусов наследования и получения преимуществ полиморфизма при компиляции. Дополнительно это выражено в использовании специальных структур данных памяти для SIMD-инструкций.

Практическая значимость состоит в создании инструмента, позволяющего проводить масштабные эксперименты с КА в реальном времени на доступном оборудовании, снижая затраты на разработку процедурного контента или моделировании различных процессов.

При выполнении работы использовались методы низкоуровневой оптимизации кода (SIMD, cache-friendly структуры данных), шаблонного программирования (CRTP), а также методы профилирования быстродействия для последующего анализа.

Выпускная квалификационная работа состоит из следующих разделов:

- аналитический раздел – анализ предметной области, рассмотрение существующих аналогов, постановка задачи;
- проектно-конструкторский раздел – проектирование и разработка библиотеки;
- экспериментальный раздел – сравнение производительности, анализ результатов, создание руководства пользователя.

1 Аналитический раздел

1.1 Библиотеки в программировании

Библиотека – набор готовых функций, классов и объектов для решения каких-то задач [1]. Их использование позволяет использовать уже написанный код, то есть повторяет принцип DRY (Don't Repeat Yourself). Например, используя библиотеку Requests в языке программирования Python, разработчик сможет написать запрос к серверу за несколько строк. Без использования этой базовой библиотеки, разработчику пришлось бы вручную писать весь код для запроса и объём кода бы кратно вырос.

Библиотеки бывают открытыми, коммерческими и закрытыми [2]. Открытые создаются сообществом и свободно распространяются. В зависимости от лицензии, можно свободно редактировать, дополнять, предлагать решения или сообщать об ошибках.

Коммерческие библиотеки разрабатываются компаниями с целью последующей продажи или для внутреннего использования (то есть не продаются), а исходный код защищён авторским правом. Например, такой библиотекой является Qt.

Закрытые библиотеки (проприетарные) являются полной противоположностью открытых библиотек: нет доступа к исходному коду, нет возможности редактировать и дополнять код. Доступный максимум взаимодействия – сообщить об ошибках на сайте разработчика. Закрытость библиотеки используется в основном для защиты авторских прав и ограничения внешнего доступа к исходному коду. Ярким примером таких библиотек являются: DirectX, NVIDIA cuDNN, Windows API.

Библиотеки в компилируемых языках разделяются на два типа: статические и динамические. Код статических библиотек встраивается в исходный код проекта, например, такая библиотека как Boost для C++, распространяется в виде набора заголовочных файлов и при компиляции

встраивается итоговый скомпилированный код. Динамические же работают иначе – они требуют загрузки уже скомпилированного кода в запущенный процесс, то есть они не встраиваются в исходный код, а лишь дополняют его. В данной работе используется подход с динамическим подключением, так как этот вариант позволяет легко интегрировать библиотеку для моделирования клеточных автоматов, в конечное приложение, а также заменять и обновлять её без обновления клиентского кода.

Использование уже написанных библиотек позволяет переиспользовать уже написанный код, что сокращает время разработки. Библиотека по моей теме работы позволяет разработчикам использовать уже оптимизированные структуры данных и алгоритмы, которые им пришлось бы писать самим.

Динамические библиотеки загружаются в то же адресное пространство, что и вызвавший код. Это позволяет беспрепятственно обмениваться данными без копирования и лишних накладных расходов, что является ключевым фактором в разработке производительной библиотеки. Именно поэтому была выбрана динамическая библиотека (DLL в Windows).

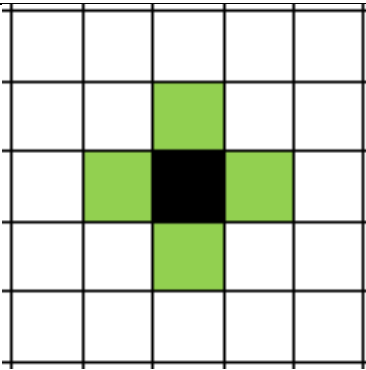
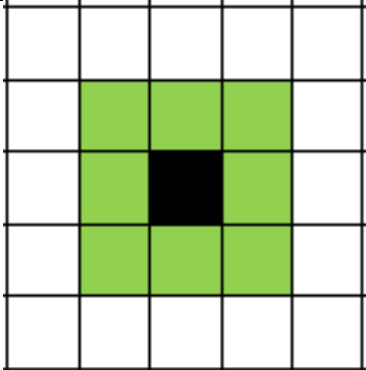
1.2 Характеристика клеточных автоматов и сферы применения

Клеточные автоматы – система, где каждая клетка на сетке зависит от состояния соседних и на основе простых правил может генерироваться очень сложное и разнообразное поведение. Например, на правилах B3 и S23 работает игра «жизнь», которая интересна своей полнотой по Тьюрингу, а при правилах B5678 и S45678 генерируются стабильные структуры, очень похожие на пещеры, которые могут использоваться в процедурной генерации [3; 4].

При изменении правил можно добиться различных конечных результатов, чем активно пользуются в науке: в физике моделируются переходные процессы (диффузия, течение жидкости), в социологии моделируются эпидемии, сегрегации, распространении паники) и многих других областях (например, моделирование распространения огня в лесных массивах сложно отнести к конкретной области).

Окрестности (соседство) может быть представлено различными формами: фон Неймана, Мура, расширенные, размерности 1D, со сдвигом, сегментированные и так далее. В работе будут использоваться окрестности фон Неймана и Мура, их описание представлено в таблице 1, где чёрная клетка – центр, относительно которого считается соседство, а зелёная – клетка является соседом.

Таблица 1 – Описание окрестностей фон Неймана и Мура

Графическое представление	Название окрестности	Описание
	фон Неймана	Соседями считаются только близлежащие клетки слева, справа, снизу, сверху.
	Мура	Соседями считаются все клетки в радиусе 1, то есть: сверху слева, сверху, сверху справа, справа, снизу справа, снизу, снизу слева, слева.

1.3 Формулировка проблемы

Несмотря на кажущуюся простоту клеточных автоматов, его реализация для высокой размерности сетки (10^4 на 10^4 клеток и более) сталкивается с серьезными проблемами производительности из-за низкой эффективности алгоритмов. Для достижения минимально допустимой частоты кадров

(принятого за 60 в компьютерной индустрии), одна итерация должна быть обработана менее чем за 16,6 мс, что для существующих подходов является редкостью.

Скриптовые языки традиционно могут иметь низкую производительность из-за интерпретатора или сборщика мусора. Компилируемые языки, наряду с скриптовыми могут так же неэффективно использовать процессорные мощности и работать с памятью, что приводит к «узким местам» при повышении размерности сетки симуляции КА. Поэтому для повышения производительности библиотеки требуется провести анализ полезных данных о процессоре.

1.4 Анализ существующих аналогов

Библиотека для клеточных автоматов представляет собой программное обеспечение для обработки входного набора данных (или генерации) и последующей обработки. На рынке было найдено следующее ПО:

1) CAME&L (в частности CADLib) — специализированная библиотека для клеточных автоматов. Имеет несколько реализаций, а также сравнительную таблицу производительности реализаций. Дата выпуска: 2003 год. Написан на C++.

2) CellPyLib — библиотека Python для работы с клеточными автоматами. Он обеспечивает краткий и простой интерфейс для определения и анализа одномерного и двумерного КА. КА может состоять из дискретных или непрерывных состояний. Радиусы окрестностей можно регулировать, а в двумерном случае поддерживаются окрестности Мура и фон Неймана.

3) MCell — программа, которая использует пространственно реалистичные трехмерные клеточные модели и специализированные алгоритмы Монте-Карло для моделирования движений и реакций молекул внутри клеток и между ними — клеточную микрофизиологию. Написан на Python.

4) Golly — кроссплатформенное приложение с открытым исходным кодом для изучения «Игры жизни» Конвея и многих других типов клеточных автоматов. Поддерживает Lua и Python, написан на C++.

Представленное выше ПО в основном написано на C++ или Python и распространяется на бесплатной основе. Имеют следующие недостатки для прикладного программирования:

- требуют подключения тяжелых зависимостей и сложных сред выполнения, что неприемлемо для легковесных приложений или игр;
- отсутствуют готовые, легко подключаемые динамические библиотеки (DLL) с бинарным API (ABI), которые можно было бы быстро интегрировать в существующий проект без глубокой переработки архитектуры;
- решения не предоставляют информации о использовании современных инструкций SIMD (SSE / AVX2 / AVX-512), а также о использовании оптимизированных алгоритмов обработки, который бы позволил максимально утилизировать ресурсы современных процессоров;

1.5 Актуальность разработки

Возникает потребность в разработке специализированной динамической библиотеки на C++, ориентированной исключительно на максимальную производительность вычислений клеточных автоматов.

Актуальность темы обусловлена следующими факторами:

- создание универсального модуля, который разработчики смогут подключать к своим проектам (играм, симуляторам) как «черный ящик», получая высокую скорость работы «из коробки», то есть получить практическую ценность;
- применение современных стандартов C++ 20/23, включая концепты (concepts) для типобезопасности и модули для структуры кода, в

сочетании с ручной оптимизацией через SIMD-инструкции обеспечивает технологическую новизну;

— предоставление готовой динамической библиотеки (DLL) упрощает взаимодействие между компонентами системы, написанными на разных языках или компилируемыми разными компиляторами, что решает проблему интеграции.

1.6 Необходимая информация о CPU

CPU, он же процессор, выполняет вычисления. В современных реалиях имеет следующие ключевые особенности, необходимые для понимания и последующей оптимизации программного продукта:

1) параллелизм на уровне инструкций (ILP – Instruction Level Parallelism) – возможность выполнять инструкции параллельно, если результат выполнения второй инструкции не зависит от выполнения первой. Современные процессоры гиперскалярны, то есть выполняют более одной инструкции за такт. Позволяет повысить производительность при правильной декомпозиции задачи на независимые блоки;

2) параллелизм на уровне данных (DLP – Data Level Parallelism) – возможность обрабатывать несколько элементов одним набором инструкций. То есть позволяет одним проводить манипуляции уже с набором данных, что должно увеличить пропускную способность данных;

3) блок предсказания ветвления (BPU - Branch Prediction Unit) - специальный блок процессора, отвечающий за предиктивную загрузку инструкций в блоках ветвления. Если в горячем цикле будет находиться большое количество блоков `if ... else`, это может негативно сказаться на времени выполнения, так как если BPU ошибётся, весь конвейер процессора будет сброшен и произойдёт загрузка инструкций в соответствии с ассемблерным кодом, что понесёт на себе дополнительные задержки, около 20 тактов [5];

4) L1 Data Cache (далее L1D) – самая быстрая память (за исключением регистров самого процессора) и ближайшая память. В среднем время доступа около одной наносекунды, а размер – 32 КБайта. При эффективном использовании данного кэша можно добиться высокой производительности и уменьшению простоя CPU, то есть добиться большей утилизации мощностей. Это достигается уменьшением количества обращений к L2 (незначительное увеличение времени доступа), L3 (значительное увеличение времени), ОЗУ (огромные задержки);

5) L2 Unified Cache – относительно быстрая память. Время доступа около 3 наносекунд, а средний размер 512 КБайт. Имеет аналогичное объяснение важности, что и L1 Data Cache;

6) L3 Unified Cache – самая большая, но и самая медленная память, расположенная на кристалле процессора. Среднее время доступа – 10 нс. Размер может сильно варьироваться – от его отсутствия в старых офисных процессорах до нескольких гигабайт для серверных решений. Хранит большое количество информации, но эта информация – общая для всех ядер процессора. Если необходимые данные не помещаются в L1D и L2, то L3 является спасением от поиска в ОЗУ;

7) ОЗУ – далёкая и не очень быстрая память. В зависимости от поколения (например, DDR4) можно получить в среднем 100 нс времени доступа. Размер ограничен лишь способностями контроллера памяти процессора;

8) SSE/AVX – специальный набор аппаратных и программных решений для векторизации вычислений для архитектуры x86_64. Позволяет использовать параллелизм на уровне данных (128 бит для SSE и 256/512 бит для AVX);

При эффективном использовании аппаратной среды и особенностей процессора, можно кратно увеличить производительность программных продуктов без замены аппаратной среды на более дорогую и производительную.

Выводы по разделу

Выбрана модель распространения библиотеки – в виде динамической библиотеки, что обеспечивает простоту интеграции во внешние приложения.

Клеточные автоматы являются эффективным решением для моделирования физических, химических и общественных процессов, однако их применение на практике ограничено вычислительной сложностью. Существующие программные аналоги не обеспечивают необходимой производительности из-за отсутствия низкоуровневых оптимизаций или имеют архитектуру, непригодную для интеграции в сторонние проекты в виде легковесных модулей.

При анализе архитектуры современных процессоров было выявлено, что эффективное использование кэш-памяти (L1, L2, L3), ILP и DLP, а также переход к SIMD-алгоритмам благоприятно повлияет на производительность динамической библиотеки.

Также была выявлена потребность в моделировании сложных систем, которые могут быть реализованы с помощью клеточных автоматов, но на рынке отсутствуют решения, которые бы позволили легко интегрировать высокопроизводительное решение в уже существующий проект простым способом без значительных затрат в разработке. Это подтверждает необходимость разработки специализированной динамической библиотеки на C++, ориентированной на максимальную производительность.

Были сформированы требования к разрабатываемому решению: обеспечение ABI для интеграции в проект, безопасность типов благодаря использованию современных стандартов C++.

2 Проектно-конструкторский раздел

2.1 Выбор средств разработки

2.1.1 Выбор языка программирования

Для реализации динамической библиотеки для моделирования клеточных автоматов был выбран язык C++. Данный выбор обусловлен необходимостью обеспечения максимальной производительности при минимальных накладных расходах времени выполнения (runtime overhead). Необходим низкоуровневый контроль за памятью и аппаратными инструкциями процессора.

Статическая типизация позволяет выявлять ошибки на этапе компиляции, что критически важно для надёжности библиотеки, а отсутствие виртуальной машины или сборщика мусора гарантирует стабильное время выполнения операций, необходимое для работы.

C++, в отличие от Python/Go/C#/Java, не имеет сборщика мусора, что не накладывает дополнительных накладных расходов на существование библиотеки. Так же использование C++ позволяет скомпилировать код в нативный, понятный для любого X86_64 процессора, минуя этапы интерпретации (например, как в Python). Под данные критерии попадает и Rust, но его относительно малая популярность и сложность перехода после C++ ставят под сомнение его валидность использования в данном ПО.

2.1.2 Выбор стандарта языка программирования

В качестве целевой спецификации языка выбрана редакция ISO/IEC 14882:2024 (C++ 23). Использование современного стандарта продиктовано следующими необходимыми для разработки факторами:

- улучшенная поддержка constexpr и compile-time вычислений, что позволяет перенести часть вычислений на время компиляции, а не на время выполнения;

- поддержка модулей (modules), которые увеличивает гибкость и простоту разработки отдельных частей библиотеки, а также улучшает время компиляции всего проекта. Все основные компоненты системы являются модулями, что значительно упрощает разработку;
- улучшенная поддержка концептов (concepts), это мощное средство для наложения ограничений на шаблонные параметры. Делает код более выразительным, кратким и удобным для понимания. Используется для ограничения разрешённых типов для шаблонных структур и функций.

2.1.3 Использование SISD и SIMD

Для увеличения производительности используется переход от скалярных вычислений (SISD [6]) к векторным – SIMD. Вектор подразумевает собой некий массив данных, где операции проводятся одновременно со всеми элементами, а не с каждым отдельно. Это позволяет кратно повысить производительность. Наглядный пример разницы работы SIMD от SISD представлен на рисунке 1.

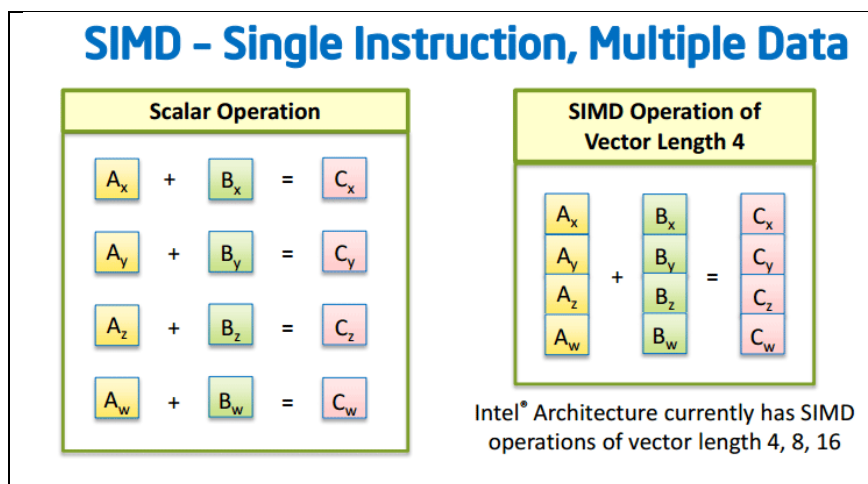


Рисунок 1 – Разница в операциях между SISD и SIMD, представленный на сайте Intel

C++ предоставляет встроенные заголовки и их функции (в данном случае Intrinsics). Используемые функции компилируются в нативные инструкции процессора, что является наилучшим вариантом, так как

отсутствуют какие-либо прослойки между итоговым кодом и процессором. Также это даёт полный контроль над данными (в частности, над выравниванием, так как это очень важный параметр для SSE/AVX инструкций). Использовался официальный гайд от компании Intel по Intrinsics X86_64 [7].

2.1.4 Выбор среды разработки

В качестве основной среды разработки был выбран программный продукт от Microsoft – Visual Studio 2026 с компилятором MSVC v145, а также MSVC Build Tools v14.51. Данный выбор обусловлен большим функционалом, поддержкой выбранного языка, а также набором необходимых инструментов для разработки. Встроенный IntelliSense помогает с форматированием, поддержкой, рефакторингом и написанием нового кода, что благоприятно сказывается на скорости разработки. Интеграция инструментов ИИ-ассистентов (GitHub Copilot) позволяет ускорить рутинные операции генерации кода и первичного анализа синтаксических ошибок, повышая общую продуктивность разработчика. Также явным преимуществом выделяется встроенный санитайзер адресов, который анализирует код и не позволяет выйти за пределы разрешённой памяти. Данный набор инструментов позволяет быстро разрабатывать безопасный код.

2.1.5 Выбор операционной системы

В качестве операционных систем, которые поддерживают текущую реализацию библиотеки, выступают Windows 11 и Windows 10. Ограничение обусловлено ограниченностью выпуска основной среды разработки – Visual Studio выпускается только под управлением ОС Windows. Архитектура спроектирована с учетом переносимости (использование стандартного C++ и интринсиков, доступных в GCC/Clang), что позволит в будущем добавить поддержку Linux/macOS без существенной переработки ядра.

2.2 Архитектура библиотеки

2.2.1 Способ взаимодействия и работа с памятью

Поскольку разрабатываемая библиотека предназначена для использования в качестве динамически подключаемого модуля (DLL библиотека) различными клиентскими приложениями, ключевым аспектом архитектуры является способ организации обмена данными между библиотекой (самим ядром, что выполняет вычисления) и вызывающим кодом.

В виду того, что библиотека и клиентское приложение выполняются в рамках единого пространства операционной системы (при загрузке динамической библиотеки вызывающим кодом), решение использовать разделяемую память (shared memory) было признано избыточным и сложным. Вместо этого был реализован механизм передачи указателя на непрерывную область памяти.

Для управления памятью был написан собственный аллокатор, который гарантирует выровненность памяти для работы с SIMD-алгоритмами. Код фрагмента выровненного аллокатора представлена на рисунке 2.

```
class AlignedAllocator {
public:
    inline static void* allocate(size_t size, size_t alignment = 16) noexcept {
        if (size == 0) return nullptr;
        if (alignment == 0 || (alignment & (alignment - 1)) != 0) {
            alignment = 16;
        }
        void* ptr = _aligned_malloc(size, alignment);
        return ptr;
    }

    inline static void deallocate(void* ptr) noexcept {
        if (ptr) _aligned_free(ptr);
    }
};
```

Рисунок 2 – Фрагмент кода кастомного аллокатора

Архитектура системы построена по принципу предоставления строго определённого бинарного интерфейса. Внешний вызывающий код взаимодействует исключительно через экспортируемые функции, что обеспечивает слабую связность и позволяет заменять внутреннюю логику библиотеки без замены клиентского кода.

Была выделена следующая наиболее эффективная последовательность взаимодействий:

- 1) вызывающий код обращается по ABI к библиотеке, например, вызывает функцию для создания игры «жизнь» размерами 256 на 256 клеток, со стандартными правилами, а также выполнением 100 итераций клеточного автомата. Дополнительно нужно передать указатель на возвращаемый результат, так как функция вернёт код ошибки;

- 2) библиотека выполняет функцию всего за одно переключение контекста (каждое переключение дорого с точки зрения циклов процессора), то есть все вышеописанные задачи будут выполнены в разном, без возвращения управления вызывающей стороне при каждой итерации;

- 3) после выполнения вычислений библиотека редактирует указатель в структуре, который указывает на получившееся поле, а также размер одного элемента (что используется для валидации вызвавшим кодом);

- 4) после обработки полученных данных, вызвавший код явным образом вызывает функцию очистки памяти, передав в качестве аргумента полученную структуру;

Диаграмму последовательности взаимодействия с библиотекой через ABI можно увидеть на рисунке 3.

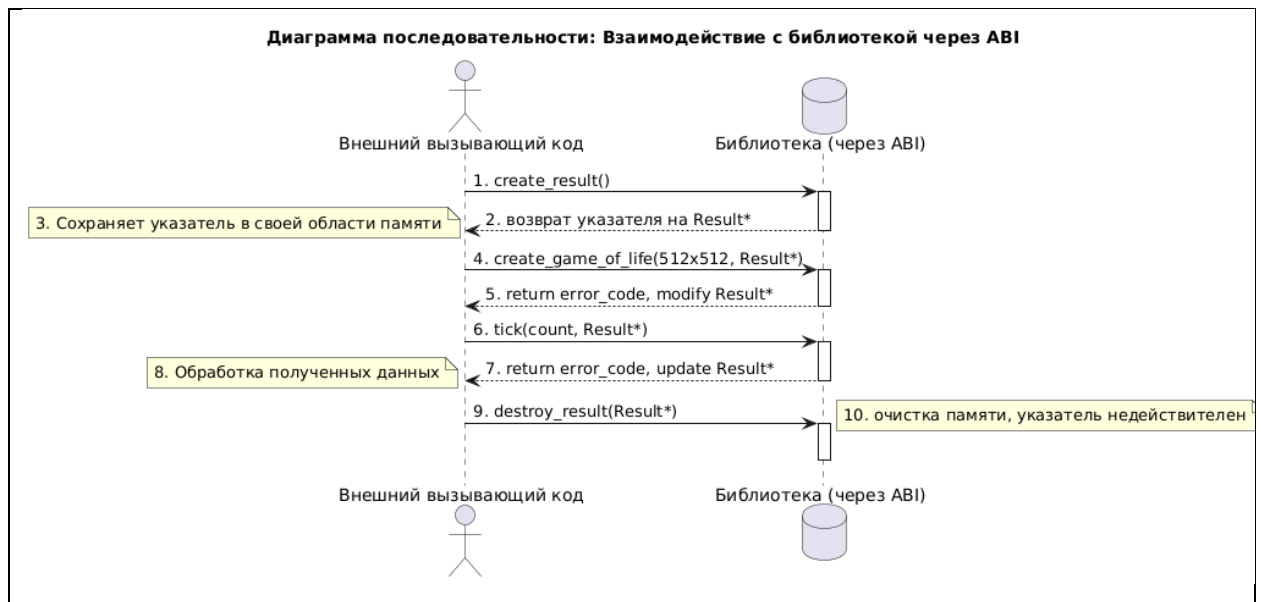


Рисунок 3 – Диаграмма последовательности взаимодействия с библиотекой

Данная архитектура позволяет исключить тяжеловесные и долгие операции, такие как сериализация данных и межпроцессные взаимодействия. Также обеспечивает максимальную производительность без лишних слоёв абстракций и накладных расходов.

2.2.2 Архитектура

Разработанная библиотека реализует многоуровневую (слоистую) архитектуру, направленную на разделение ответственности между интерфейсом взаимодействия, логикой диспетчеризации и аппаратно-зависимыми реализациями алгоритмов. Графическое представление архитектуры представлено на рисунке 4.

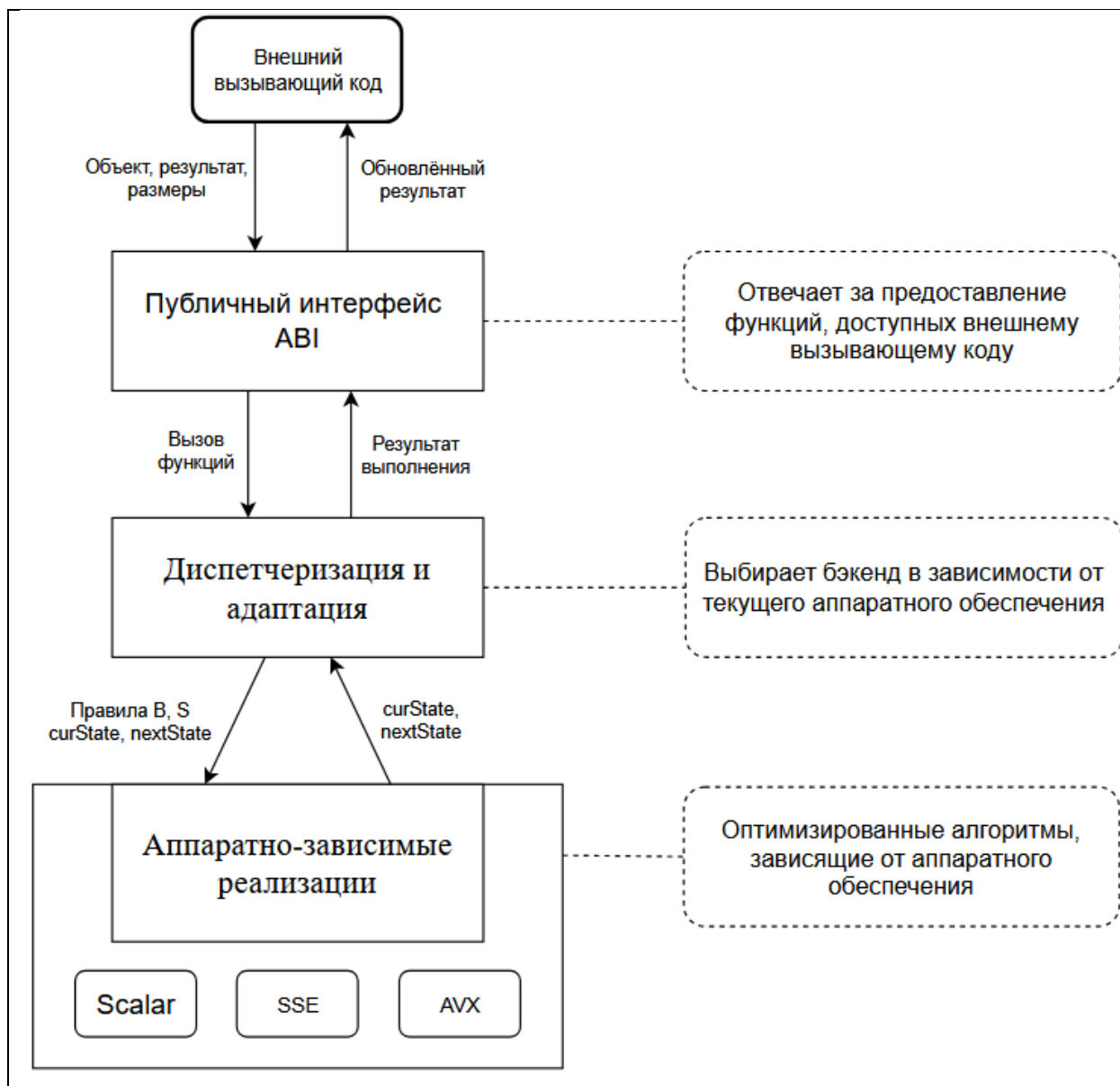


Рисунок 4 – Графическое представление архитектуры библиотеки

Архитектура состоит из трех основных уровней:

- 1) публичный интерфейс;
- 2) диспетчеризация и адаптации (middleware);
- 3) аппаратно-зависимые реализации.

Подробное описание каждого уровня будет представлено в пунктах 2.3.1, 2.3.2 и 2.3.3 соответственно.

2.3 Алгоритмы и структуры данных

2.3.1 ABI – интерфейс бинарного взаимодействия

Верхний уровень архитектуры библиотеки представляет собой Application Binary Interface (ABI). Этот слой служит единственной точкой входа для внешних приложений и полностью инкапсулирует внутреннюю сложность реализации, включая управление памятью, выбор бэкендов и логику клеточного автомата.

При разработке интерфейса были использованы следующие принципы:

- использование только простых типов данных, имеющих однозначное представление в памяти на всех платформах (int, float, указатели, размеры size_t, то есть Plain Of Data). Это позволяет вызывать функции библиотеки из любых языков программирования, поддерживающих Foreign Function Interface (FFI) при соглашении вызовов cdecl, таких как Python (ctypes/cffi), Java (JNI) или C# и других;
- отсутствие исключений C++, все экспортируемые функции помечены как extern "C" и не выбрасывают исключений (noexcept), а ошибки передаются через коды возврата;
- внутренние объекты (контекст симуляции, буферы данных) скрыты от клиентского внешнего доступа.

Так как ABI это входной интерфейс, в нём должны быть включены все проверки на валидность данных. Если произошла ошибка валидации - вернёт код ошибки (0 при отсутствии).

Интерфейс разделён на 2 логические группы функций:

- 1) управление жизненным циклом клеточного автомата;
- 2) выполнение симуляции.

Управление жизненным циклом отвечает за создание и удаление клеточного автомата с заданными параметрами. Например, функции NewGameOfLife нужно передать указатель на результат выполнения, возвратит код ошибки. Для очищения памяти потребуется вызвать

DeleteGameOfLife и передать в качестве аргумента полученный указатель на результат.

Для работы с клеточными автоматами предусмотрены функции:

- 1) NewResult без параметров, он вернёт указатель на структуру с результатами выполнения, которую нужно использовать далее;
- 2) DeleteResult, но он не может быть вызван из вне. Он вызывается автоматически после вызова DeleteGameOfLife или DeleteCellularAutomaton;
- 3) NewCellularAutomaton с параметрами:
 - a. size_t width;
 - b. size_t height;

Возвращает структуру с результатами выполнения.

4) SetRulesB и SetRulesS требуют указатель на клеточный автомат. Имеет вариант с установкой через диапазон чисел от RulesBFrom до RulesBTo включительно. Для правила S аналогично;

5) DeleteCellularAutomaton, требует передать указатель для очищения данных.

Для выполнения симуляции предусмотрена функция Tick. В качестве аргумента передаётся указатель на клеточный автомат и количество тиков (для оптимизации количества переключения контекстов). Возвращает структуру на результат выполнения.

Данная архитектура позволяет исключить повторное выделение памяти для копирования результатов в вызывающий код, упростить взаимодействие (не потребуются дополнительные преобразования для дальнейшей работы с данными).

2.3.2 Middleware – диспетчер реализаций

Middleware (промежуточный слой) отвечает за связывание высокоуровневого интерфейса библиотеки с низкоуровневыми аппаратно-зависимыми бэкендами. Его главная задача — обеспечить динамический

выбор оптимальной реализации во время выполнения программы без потери производительности внутри вычислительных циклов. Middleware реализует паттерн Strategy в комбинации с Factory.

При создании экземпляра CellularAutomaton или контекста симуляции выполняет детекцию возможностей процессора:

- если поддерживается AVX2 – создается экземпляр AVX2Backend;
- если поддерживается SSSE3 – создается экземпляр SSEBackend;
- иначе – создается экземпляр ScalarBackend.

Блок схема алгоритма выбора реализации бэкенда в зависимости от аппаратной поддержки процессором представлен на рисунке 5. Фрагмент кода класса, который возвращает поддерживаемые технологии процессора во время выполнения представлен на рисунке 6.

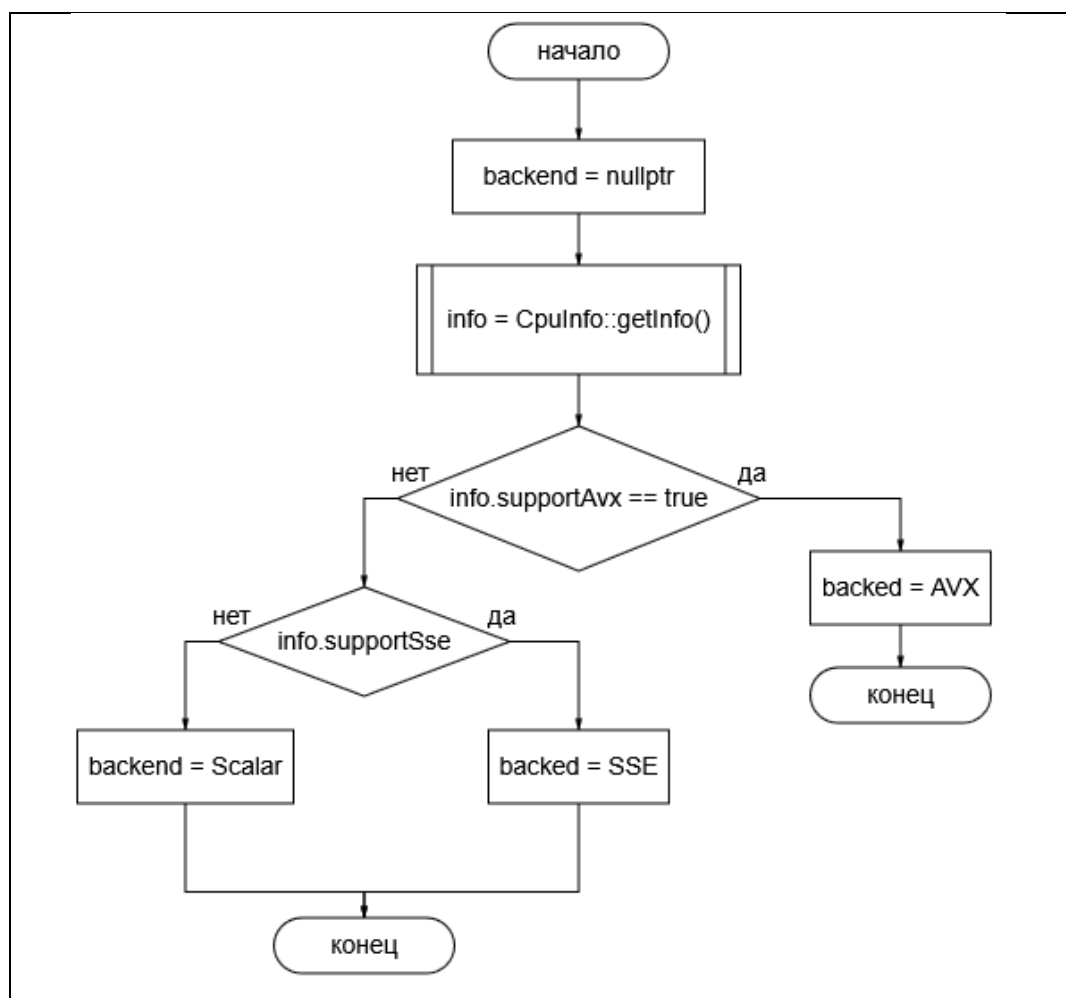


Рисунок 5 – Блок схема алгоритма выбора бэкенда

```

public:
    struct CpuInfo {
        bool supportAvx;
        bool supportSSE;
    };

    static CpuInfo getInfo() {
        return CpuInfo(AVX2(), SSSE3());
    }

    static bool SSSE3(void) { return CPU_Rep.f_1_ECX_[9]; }
    static bool AVX2(void) { return CPU_Rep.f_7_EBX_[5]; }

```

Рисунок 6 – Фрагмент кода, указывающий на поддержку нужной технологии процессором

Выбранный бэкенд сохраняется в виде указателя на абстрактный интерфейс. Внутренняя структура бэкенда использует CRTP, но для внешнего мира он представлен как объект, реализующий метод `Tick()` [8].

При вызове метода `Tick()`, `Middleware` не выполняет вычисления сам, а возвращает указатель на необходимый аппаратно-зависимый бэкенд, который и занимается вычислениями. А поскольку тип бэкенда известен объекту `Middleware`, происходит переход к конкретной реализации.

`Middleware` необходим из-за сложностей с выбором алгоритма под текущую аппаратную систему: разработчик может использовать библиотеку как на современном процессоре с поддержкой AVX512 (в основном серверные решения, недоступные обычному потребителю), так и с поддержкой AVX-2. Так же, он может работать со старым процессором, который поддерживает только SSSE3. В случае совсем старой конфигурации выбирается обычный скалярный метод. Без этого компонента системы, приходилось бы включать в `hot path` большое количество блоков `if... else` для выбора подходящего алгоритма, что значительно бы увеличило время работы [5].

Каждый различный алгоритм изолирован в отдельном классе и изменение одного алгоритма не сломает другой, что упрощает поддержку и тестирование.

Дополнительно это даёт гибкость выбора реализации – она происходит динамически и скорость выполнения. Используется CRTP и статическая диспетчеризация. Таким образом, накладные расходы на полиморфизм возникают только один раз при входе в функцию Tick (переход к указателю на функцию), а миллионы итераций цикла выполняются с максимальной скоростью, так как компилятор видит весь код конкретного бэкенда целиком и может применить агрессивные оптимизации, недоступные при недостаточном уровне знания всей цепочки вызовов.

2.3.3 Вычисление суммы соседей и применение правил. Аппаратно-зависимые бэкенды

Основной операцией в клеточном автомате является подсчёт количества живых (активных) соседей для каждой клетки по окрестности Мура (8 соседей). Блок схема алгоритма подсчёта суммы соседей (количества соседей) представлена в приложении А.

Пошаговый алгоритм для общего случая для окрестности Мура:

- 1) для каждой клетки (x, y) текущего поколения берётся окрестность 3×3 ;
- 2) соседи с координатами $(x \pm 1, y \pm 1)$ суммируются, за исключением самой клетки (x, y) ;
- 3) если соседняя клетка выходит за границы, она не учитывается;
- 4) результат — целое число от 0 до максимального значения типа, которое используется для определения следующего состояния клетки.

Для выполнения подсчёта предусмотрено несколько основных реализации (таких как наивная, скалярная и использующая SSE/AVX). Главное требование к реализации — они возвращают одинаковый результат.

Предусмотренные реализации отличаются по времени выполнения и возможности выполнения на конкретной аппаратной системе:

- самая простая (наивная), работающая на всех системах и процессорах, где есть блок ALU, то есть на любом персональном компьютере. Реализация отличается своей медлительностью;
- скалярная использует оптимизированный алгоритм. Работает так же на всех системах. Значительно повышает производительность;
- SSE, то есть векторизированный алгоритм должен работать только на процессорах с поддержкой SSE2/SSSE3. Ожидается высокая производительность за счёт параллелизма на уровне данных (выполняются операции над целым блоком данных 128 бит, а не над одиночным числом). Имеет значительную сложность разработки;

– AVX, второй векторизованный алгоритм. Работает только на процессорах с поддержкой AVX-2. Пиковая производительность теоретически должна быть в два раза выше SSE из-за увеличения размера регистров до 256 бит.

Последовательный алгоритм применения правил В и S:

- 1) вычисляется количество соседей n клетки с координатой x и y ;
- 2) если клетка мертва (то есть значение $\text{curState}[x, y] == 0$) и $n \in B$, то клетка становится живой;
- 3) если клетка жива ($\text{curState}[x, y] == 1$) и $n \in S$, то клетка остаётся живой;
- 4) иначе клетка становится мёртвой.

Блок схема алгоритма применения правила В и S представлен на рисунке 7, где $\text{curState}[x, y]$ и $\text{nextState}[x, y]$ – значения текущего и следующего полей клеточного автомата, n – количество соседей, В и S – правила рождения и выживания.

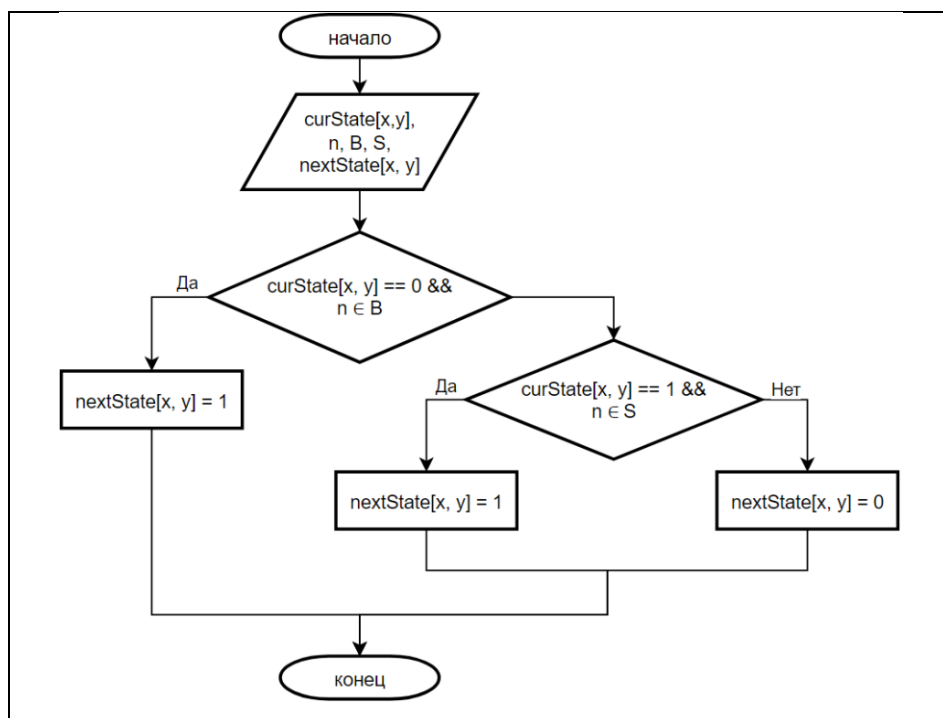


Рисунок 7 – Блок схема алгоритма применения правила В и S

Процесс обновления сетки зависит от реализации бэкенда. Общий алгоритм:

- 1) подсчёт соседей для всех клеток (с использованием выбранной реализации);
- 2) применение правил B/S к каждой клетке;
- 3) запись результата в буфер следующего шага.

Для векторизированных алгоритмов есть смысл использовать векторизованные алгоритмы применения правил B/S к полученному массиву, так как в этом случае минует запись во временный буфер или `nextState`, что уменьшает время работы, но данное утверждение требует экспериментальной проверки.

Все реализации инкапсулированы в отдельные классы, что позволяет использовать их через единый интерфейс, реализованный с помощью шаблонного паттерна CRTP, обеспечивая гибкость архитектуры и избегая виртуальные вызовы.

2.3.4 Реализация алгоритмов вычисления суммы соседей

Было реализовано 5 алгоритмов вычисления суммы соседки по окрестности Мура. Все они представлены в листингах приложения Б:

- Б.1, модуль реализации наивного (самого простого) алгоритма. Использует сложение на основе АЛУ. Имеет чрезмерное использование блоков `if ... else`, что негативным образом сказывается на блоке предсказания ветвлений [9];
- Б.2, модуль скалярной реализации алгоритма. Использует совершенно иной подход к суммированию – поэлементное сложение по всей кэш-линии процессора. То есть, идёт итерация по верхней, средней и нижней строчке и вычисляется сумма. Эта поэлементная сумма вертикальных элементов складывается в отдельный буфер. Для краевых случаев используется немного модифицированный алгоритм. Для не краевых случаев

используется общий алгоритм: горизонтально складываются вертикальные суммы и вычитается середина клетки. То есть вычисляется сумма вообще всех соседей в окрестности 3×3 , и затем вычитается центральная клетка, чтобы осталась только окрестность Мура;

– Б.3, модуль оптимизированной скалярной реализации. Основан на модуле скалярного алгоритма, но убрано использование одного буфера, который хранил смещения от начала буфера клеточного автомата. Это позволяет убрать выделение памяти системой. А так как время выделения памяти имеет высокую нестабильность, это негативно сказывалось на общем времени выполнения.

– Б.4, модуль вертикальной SSE реализации. Использует SSE, итерируется сверху вниз, слева направо. Сам алгоритм проще в реализации, чем горизонтальный SSE, но заметно сложнее, чем оптимизированный скалярный вариант. Не выделяет временных буферов. Так как алгоритм итерируется по вертикали и использует окно в 14 элементов, он не может корректно обрабатывать смежные случаи. Для решения этой проблемы используется неоптимальный вариант сохранения данных после высчитывания суммы соседей – не выровненное сохранение с последующим не выровненным доступом. Данные операции сильно замедляют время работы с памятью, а также вызывают больше количество не выровненной работы с памятью, что так же негативно сказывается на общем времени работы. Дополнительным замедлением является неоптимальное использование регистра SSE: при размере в 16 элементов, в результате вычислений используются лишь 14;

– Б.5, модуль горизонтальной SSE реализации. Использует SSE, итерируется слева направо, сверху вниз. Трудностей в реализации добавляет факт сохранения промежуточных данных для вычисления текущего и следующего окна данных (окно – 16 элементов типа `byte`, `bool`, `uint8_t`). Но такая реализация позволяет полностью убрать не выровненный доступ к памяти. Дополнительно это позволяет вычислять одновременно несколько

вертикальных блоков данных: для первых строк требуется два регистра, для средних строк от трёх до 5, для последних двух строк – два регистра.

- Б.6, модуль горизонтальной AVX реализации. Базируется на горизонтальной SSE реализации, то есть наследует все его черты. Главным отличием является увеличение окна данных с 16 до 32 элементов. В идеальных условиях выполнится в два раза быстрее оригинального алгоритма.

2.3.5 Реализация алгоритмов применения правил В и S

Для алгоритма применения правил В и S было реализовано 7 различных вариантов. Их листинг представлен в приложении В и имеет описание:

- В.1, модуль с использованием SSE. Использует специальную таблицу для работы и небольшое количество инструкций. Записывает в результат массив полученных данных;
- В.2, модуль с использованием AVX. Работает аналогично SSE, но использует AVX-2 с немного облегчённой реализацией, что должно уменьшить количество тактов процессора для выполнения;
- В.3, модуль с наивной реализацией. Использует блоки if ... else. В результат сохраняет записывает одно число;
- В.4, модуль с оптимизированной реализацией. Не использует блоки if ... else, а полагается на тернарные операторы. В результат записывает одно число;
- В.5, модуль с использованием bitset. Результат записывает как одно число;
- В.6, модуль, полагающийся на оптимизированную реализацию, но сохраняющий результат в виде массива. То есть делает это аналогично SSE и AVX, записывая как массив;
- В.7, модуль, использует базу от предыдущей, но использует bitset.

2.3.6 Плоский массив

Самой главной структурой данных в данной работе является объект для хранения данных в памяти. Для хранения двумерной сетки (поля) клеточного автомата выбрана структура данных – класс `Flat2DArray`, реализующая концепцию плоского (одномерного) массива с эмуляцией двумерного доступа.

В отличие от традиционного подхода с использованием массива указателей (T^{**}) или вектора векторов (`std::vector<std::vector<T>>`), плоский массив обеспечивает непрерывное размещение данных в памяти. Это очень важно для производительности по следующим причинам:

- при обходе соседних ячеек (что является типичным паттерном доступа в клеточных автоматах) процессор загружает данные в кэш-линии эффективно, то есть является *cache-friendly*, не прыгает между разными массивами (которые надо ещё загрузить), минимизируя промахи кэша;
- при линейной адресации аппаратному предвыборщику данных проще прогнозировать следующие адреса памяти;
- непрерывный блок памяти идеально подходит для векторных операций, так как позволяет загружать данные в SIMD-регистры без необходимости сбора разрозненных фрагментов памяти;
- непрерывным блоком памяти технически проще пользоваться для передачи через ABI. При использовании массива разных блоков памяти пришлось бы реконструировать данные в единый массив, что создавало бы дополнительную нагрузку на подсистему памяти и вызывало дополнительные задержки.

Разрешённые типы данных строго ограничены, разрешены только: `int`, `float`, `uint8_t` (byte) и `bool`. Далее эти типы данных будут указаны как `T`. Данные ограничения наложены использованием SIMD, так как каждый тип данных имеет прямое соответствие к регистрам SSE/AVX: `int`, `byte` и `bool` – `__m128i`, `float` – `__m128`. При указании неправильного типа данных для регистра, произойдёт ошибка компиляции. Попытка инстанцировать (создать класс с

шаблонным параметром) Flat2DArray не из разрешённых типов приведёт к ошибке компиляции, которые предотвратят ошибки во времени выполнения. Так же это дополнительно ограничит неправильное использование при дальнейшей разработке.

Представление ограничения типа переменной в виде кода, представлен на рисунке 8.

```
// Список разрешённых типов
export template<typename T>
concept allowed_type =
    std::is_same_v<T, int> ||
    std::is_same_v<T, float> ||
    std::is_same_v<T, uint8_t> ||
    std::is_same_v<T, bool>
;
```

Рисунок 8 – Ограничение разрешённых типов T

Класс инкапсулирует сырой указатель на массив элементов типа T[] внутри умного указателя std::shared_ptr<T[]>. Использование умного указателя даёт неоспоримые преимущества в разработке – упрощение за счет слежения за циклом жизни памяти. Память освобождается автоматически, когда последний владелец объекта Flat2DArray выходит из области видимости или удаляется.

Отсутствуют накладные расходы на копирование благодаря семантике разделяемого владения, передача объекта Flat2DArray между функциями или слоями архитектуры происходит дешево (копируется только управляющий блок shared_ptr, а не сам массив данных).

Уменьшение количества точек отказа (исключений) достигается за счёт конструктора, который использует shared_ptr<T>, что обеспечивает атомарность выделения памяти под объект и управляющий блок, повышая надёжность кода.

Выделяющейся особенностью является использование кастомного аллокатора. Так как для работы с SIMD необходимо выравнивание согласно размерам регистров, используется выровненное выделение памяти. Память под массив выделяется с выравниваем 16 для SSE и 32 для AVX что позволяет использовать эффективные функции `_mm_load_si128` для SSE и `_mm256_load_si256` для AVX, вместо использования не выровненных версий `mm_loadu_si128` и `mm256_loadu_si256`. Не выровненные версии могут быть медленнее из-за дополнительных операций переупаковки памяти чем выровненные [10].

Размер выделяемой памяти зависит от переданных размеров и размеру типа T. Для массива 512*512 типа `bool` будет выделено $512*512*\text{sizeof}(\text{bool})$ последовательных байт. В данном случае выделенный размер будет зависеть от машины (например, `bool` не обязательно занимает 1 байт, может и несколько, так как стандартом это не закреплено) и от выравнивания. Есть особенность хранения типа `bool` стандартным контейнером `std::vector` - данные хранятся в виде упакованной прокси структуры, напоминающей тип `uin8_t`, что обеспечивает хранение 8 переменных в одной ячейке памяти. Это увеличивает эффективность использования памяти в 8 раз, но также добавляет некоторые расходы на выбор нужного бита (то есть значения переменной). В случае с плоским массивом применение данной особенности только навредит работе алгоритма, так как потребует предварительной работы с массивом (распаковка бит для использования).

Интерфейс класса и его сигнатуру можно увидеть на рисунке 9. Проектирование с учетом требований к высокой производительности привело к реализации следующих методов и полей:

- 1) метод доступа `at(x, y)` и `operator[]`:
 - реализованы как `inline` функции с атрибутом `noexcept`;
 - преобразование координат `(x, y)` в линейный индекс выполняется по формуле $\text{index} = y * \text{width} + x$;

- отсутствие проверок границ устраняет ветвления в горячих циклах;
- 2) метод `data()` возвращает сырой указатель `T*` на начало буфера;
- 3) поля метаданных (`width`, `height`, `capacity`) позволяет внешнему коду и внутренним алгоритмам знать геометрию сетки без необходимости хранить эти данные у себя, то есть позволяет убрать дублирование данных;
- 4) методы отладки и вывода. Реализация оператора вывода `operator<<` позволяет интегрировать структуру в системы логирования и отладки, обеспечивая человекочитаемое представление состояния сетки без нарушения инкапсуляции.

```
export
template<typename T> requires allowed_type<T>
class Flat2DArray {
private:
    shared_ptr<T[]> _array = nullptr;
    size_t _width = 0, _height = 0;
public:
    Flat2DArray() noexcept;
    Flat2DArray(size_t width, size_t height, size_t alignment) noexcept;
    Flat2DArray(const Flat2DArray<T>& other) noexcept;
    Flat2DArray(Flat2DArray<T>&& other) noexcept;

    inline T& at(size_t x) const noexcept;
    inline T& at(size_t x, size_t y) const noexcept;

    inline T& operator[] (size_t posX) noexcept;
    inline const T& operator[] (size_t posX) const noexcept;

    T* data() const noexcept;
    size_t capacity() const noexcept;
    size_t width() const noexcept;
    size_t height() const noexcept;
```

Рисунок 9 – Фрагмент сигнатуры класса `Flat2DArray`

Использование `std::vector<T>` в данном случае не подходит. Требуется более гибкий инструмент для работы с памятью (выравнивание, кастомный

аллокатор). Так же требуется явная работа с указателями для передачи во вне, что не совсем удобно в случае с vector.

2.3.7 Результат выполнения

Для передачи результатов симуляции от ядра библиотеки к вызывающему приложению используется легковесная структура `SimulationResult`. Данная структура спроектирована с учетом требований к бинарной совместимости и эффективности передачи больших объемов данных без их копирования. Описание структуры представлено в таблице 2.

Таблица 2 – сигнатура структуры `SimulationResult`

Поле	Тип переменной	Описание
<code>ptr</code>	<code>T*</code>	Указатель на начало данных результата
<code>width</code>	<code>size_t</code>	Ширина поля в элементах
<code>height</code>	<code>size_t</code>	Высота поля в элементах
<code>sizeofElement</code>	<code>size_t</code>	Размер элемента в байтах

Данная сигнатура позволяет получить следующие преимущества:

- передаётся только указатель на данные (`ptr`), то есть копирования не происходит. Это важно для высокой производительности, так как все операции с выделенной памятью (ОЗУ) несут в себе большие задержки;
- передача ширины и высоты поля позволяет не заботиться о хранении этих данных у себя, а лишь вызвать ABI, который вернёт готовый результат;
- размер элемента в байтах гарантирует правильность интерпретации полученного указателя к типу данных. Поле размера элемента позволяет клиентскому приложению корректно интерпретировать тип данных, возвращаемых по указателю, обеспечивая типовую безопасность на границе раздела модулей;

– данные, которые содержит указатель, выровнены согласно требованиям алгоритма и не требуют переупаковки.

Именно эта структура отвечает за передачу значений в третьем этапе взаимодействия внешнего кода с библиотекой.

2.3.8 Класс с клеточным автоматом

Программная модель клеточного автомата в рамках библиотеки инкапсулирована в классе `CellularAutomaton`. Данный класс объединяет хранение состояния сетки, конфигурацию правил перехода и механизм выполнения шага симуляции, интегрированный с системой динамического выбора аппаратной реализации (вышеописанным `middleware`). Сигнатура класса описана в таблице 3.

Таблица 3 – Сигнатура класса `CellularAutomaton`

Поле	Тип переменной	Описание
B	<code>unordered_set<int></code>	Правила рождения клетки
S	<code>unordered_set<int></code>	Правила выживания клетки
<code>curState</code>	<code>Flat2DArray<T>*</code>	Указатель на текущее поле
<code>nextState</code>	<code>Flat2DArray<T>*</code>	Указатель на поле для следующего шага

За хранение состояния отвечает система двух независимых буферов: состояние клеточного автомата представлено двумя экземплярами класса `Flat2DArray<T>`, первый используется для обозначения текущего состояния (`curState`), второй (`nextState`) для следующего шага.

Использование двух независимых буферов является требованием для корректной работы алгоритмов нахождения суммы клетки. Поскольку новое значение каждой ячейки зависит от соседей на момент итерации, запись результата непосредственно в `curState` привела бы к искажению данных для еще не обработанных ячеек. Разделение на разные состояния шага итерации

(curState и nextState) позволяет гарантировать атомарность итерации относительно состояния системы.

Логика поведения автомата определяется набором правил В (рождения) и S (выживания). Например, игра «Жизнь» Джона Конвея имеет правила В3/S23.

Правила хранятся в виде ассоциативных контейнеров STL – std::unordered_set или в std::bitset (в зависимости от реализации, так как потребуется проверить производительность каждого варианта). Это позволяет сопоставить количество соседей с поведением за короткое время. Для bitset поиск выполнится за $O(1)$. Для unordered_set в среднем за $O(1)$, а в худшем за $O(N)$. Правила в коде библиотеки имеют соответствующие названия: В и S. Дополнительно предоставлен интерфейс для установки значений:

- SetB(vector<int>& rulesB) noexcept позволяет установить правила В из массива rulesB. Так как все последующие методы не генерируют исключений, параметр noexcept будет опущен;
- SetB(initializer_list<int>& rulesB) позволяет установить правила из списка инициализации rulesB. Отличается от варианта с использованием vector'a лишь простотой установки правил для ручного тестирования;
- SetB(int rulesBFrom, int rulesBTo) устанавливает правила в диапазоне от rulesBFrom до rulesBTo включительно.

Для правила S реализован аналогичный набор методов.

Возможность менять правила позволяет изменять поведение клеточного автомата, реализовывая различные сценарии использования, в зависимости от желаний конечного пользователя.

Выполнения шага симуляции (нахождения суммы соседей, применения правил В и S) выполняется в методе Tick. Особенностью является то, что он делегирует выполнение в специализированный бэкенд посредством middleware. То есть он получает от middleware указатель на текущую реализацию бэкенда и передаёт ей выполнение (которое выполняется согласно

правилам). После выполнения итерации, нужно сменить состояния местами, так как выполнен следующий шаг. Теперь следующий шаг станет текущим, а текущее состояние будет отвечать за следующий шаг.

Преимущество такого построения класса в разделении ответственности – класс отвечает только за сам процесс, не погружаясь в вычисления. То есть он является лишь удобной и легковесной оболочкой над специализированными вычислениями. Функциональность класса можно легко поменять, расширив алгоритмы для middleware.

2.4 Бенчмарк

Для диагностики времени выполнения различных реализаций был разработан специальный бенчмарк. Возвращаемый результат замеров (структура BenchmarkResult) представлен в таблице 4.

Таблица 4 – Сигнатура результата замера

Поле	Имя	Что измеряется
name	Имя измеряемой функции	Время
avgTime	Среднее время выполнения	
stdDev	Среднеквадратичное отклонение	
min	Минимальное время	
max	Максимальное время	
p0_01	0,01 перцентиль	
p0_1	0,1 перцентиль	
p1	1 перцентиль	
p50	Медиана	
p99	99 перцентиль	
p99_9	99,9 перцентиль	
p99_99	99,99 перцентиль	
variation	Коэффициент вариации	Процент
savedAs	В какой единице измерения сохранёно (нс, мкс, мс)	-

SavedAs представлен как перечисление, где каждая единица измерения соответствует разнице во сколько раз он больше наносекунды. То есть, наносекунда – 1, микросекунда – 1000, миллисекунда – 1000000. Это сделано для того, чтобы можно было сохранять и отобразить результат в любом удобном формате. Сам же бенчмарк изначально вычисляет время выполнения

в наносекундах, как самое точное, что возможно измерить стандартными средствами, и лишь потом делит на параметр savedAs.

Сигнатура бенчмарка представлена на рисунке 10.

```
template<typename Callable>
BenchmarkResult run(size_t warmups, size_t runCount, string name,
    SaveTime saveAs, Callable&& codeToBenchmark) {
```

Рисунок 10 – Сигнатура бенчмарка

Callable – измеряемая функция или лямбда. Бенчмарк измеряет время выполнения функции runCount раз, перед этим выполнив warmups раз функцию. В качестве способа замера времени используется std::chrono::steady_clock, а так же его методы ::now() и duration_cast<nanoseconds>. Каждое измерение времени попадает в массив всех замеренных выполнений. На основе этого массива вычисляются параметры, описанные в таблице 4 (за исключением name и savedAs).

Для измерения производительности алгоритмов, представленных в пунктах 2.3.4 и 2.3.5, были написаны отдельные функция, которые использует бенчмарк. Первая функция замеряет время выполнения для поиска суммы соседей, вторая для применения правил В и S. Они обе имеют одинаковую структуру и принимают следующие аргументы:

- 1) количество разогревочных прогонов;
- 2) количество замеряемых итераций;
- 3) ширина генерируемого массива;
- 4) высота генерируемого массива;
- 5) количество генерируемых массивов;
- 6) массив выполняемых функций.

Данный набор аргументов позволяет измерять производительность для различных конфигураций размеров и количества массивов. Так как каждый алгоритм не изменяет начального состояния поля (массива), то исходный

массив будет генерироваться лишь раз при вызове. Второй массив (куда будут сохраняться результаты суммирования) для чистоты эксперимента будет копироваться для каждого прогона. То есть для каждого алгоритма будет предоставлена лишь копия.

Ниже представлена последовательность выполнения:

- 1) генерация исходных массивов;
- 2) создание массива полученных результатов;
- 3) копирование результирующего массива для передачи в функцию;
- 4) вызов бенчмарка для каждой переданной функции в соответствии со входными аргументами;
- 5) добавление результатов нормализованного бенчмарка (время выполнения, делённое на общее количество массивов, позволяет получить более общую картину) в массив полученных результатов;
- 6) вывод всех полученных результатов.

Данная структура позволяет замерить время выполнения для всех созданных алгоритмов и выводить их в удобном для анализа формате.

Выводы по разделу

Спроектирована архитектура библиотеки по принципу уровней. CRTP позволил использовать преимущества наследования без штрафов на виртуальные функции. Для хранения данных применен плоский массив с кастомным аллокатором, который гарантирует выравнивание памяти, а также позволяет эффективно использовать кэш процессора. Использование SIMD-инструкций (SSE/AVX) обеспечивает качественный рост производительности операций подсчета соседей за счет векторизации вычислений. Для аппаратных сред без поддержки поддержки SIMD, используется скалярный алгоритм.

Разработанный внешний интерфейс обеспечил простоту и безопасность работы с внешним вызывающим кодом.

Обоснован выбор языка C++ и стандарта C++23 для обеспечения высокой производительности и низкоуровневого контроля памяти.

Разработанный бенчмарк позволит оценить производительность того или иного алгоритма для дальнейшего использования и анализа.

Использование модулей позволило легко добавлять и удалять функциональные блоки (реализацию бэкендов).

3 Экспериментальный раздел

3.1 Среда и методика испытаний

В качестве среды испытаний выступает ПК со следующими характеристиками (оставлены только комплектующие, непосредственно используемые в испытаниях):

- Процессор: Ryzen 7 5700X3D (L1 Data Cache - 32 КБайт на ядро, L2 Unified Cache – 512 КБайт на ядро, частота ядер – 4.05 ГГц);
- ОЗУ: 4 модуля памяти по 8гб DDR 4 с частотой 3466 МГц.

В настройках компилятора выставлен параметр «Включить расширенный набор инструкций» на «Не задано» (то есть не будет попыток векторизировать вычисления).

Источниками данных станут разработанные бенчмарки. Корректность полученных итераций будет сравниваться с эталоном. Дополнительные данные процессора будут собираться с помощью утилиты от компании AMD – uProf.

Будет замеряться время выполнения X (в зависимости от бенчмарка) итераций алгоритма, усреднённые относительно количества массивов.

3.2 Измерение производительности для поиска сумм соседей

Используя написанный бенчмарк, для лучшей наглядности различий производительности были использованы следующие параметры:

- размеры массивов в элементах – $4096 * 4096$ (размер элемента 1 байт). Конечный размер массива больше объёма L1 и L2 кэша;
- количество массивов – 100;
- выравнивание в байтах – 32;
- количество итераций бенчмарка – 1000;
- количество разогревочных итераций – 10;
- единица измерения время выполнения – микросекунды.

Далее будут представлены результаты запусков бенчмарка. Гистограммы упростят понимание различий реализаций (их время выполнения, среднее время выполнения и так далее).

На рисунке 11 представлена гистограмма минимального, медианного, среднего и максимального времени выполнения каждого алгоритма для поиска суммы соседей по окрестности Мура.

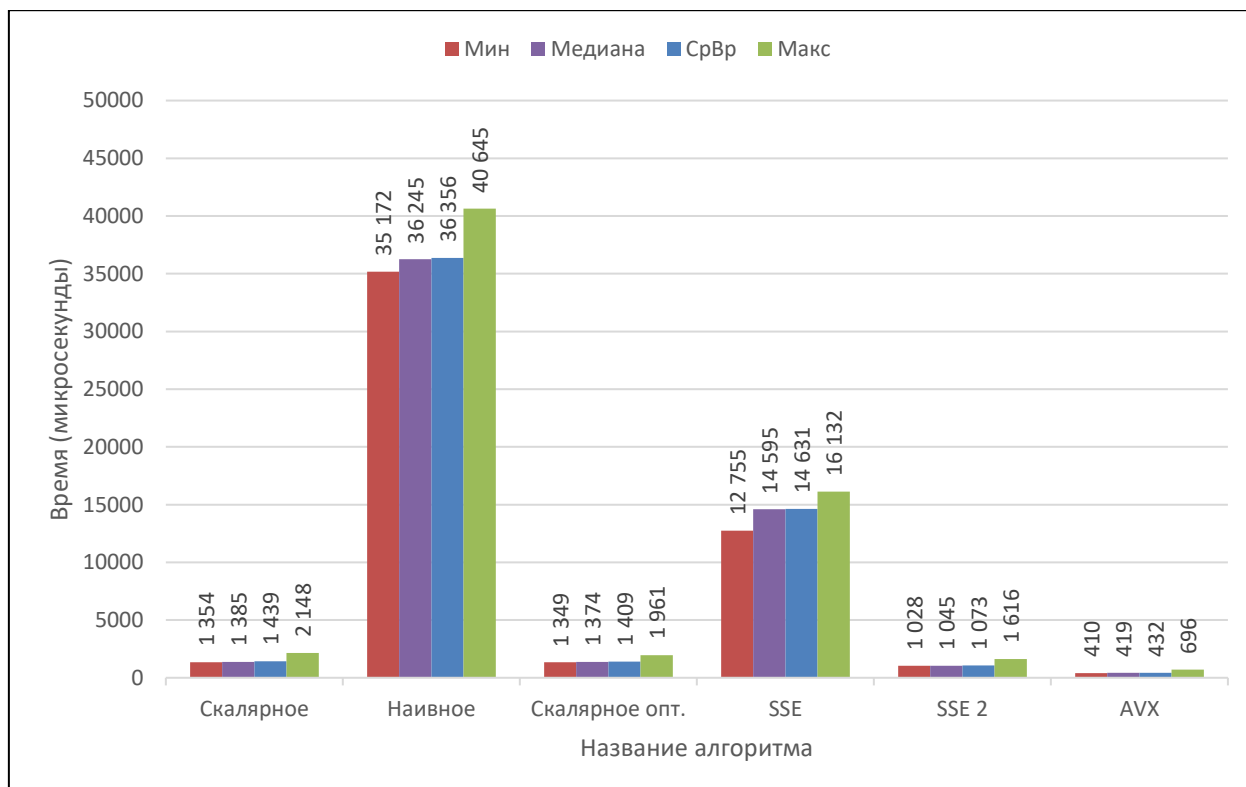


Рисунок 21 – Минимальное, медианное, среднее и максимальное время

Исходя из рисунка 11, можно выделить самое медленное решение – наивное. Далее этот алгоритм входить в графики не будет, так как имеет наихудшие значения, что значительно усложняет чтение гистограмм. Рисунок 12 содержит данные из рисунка 11 и исключает наивный алгоритм.

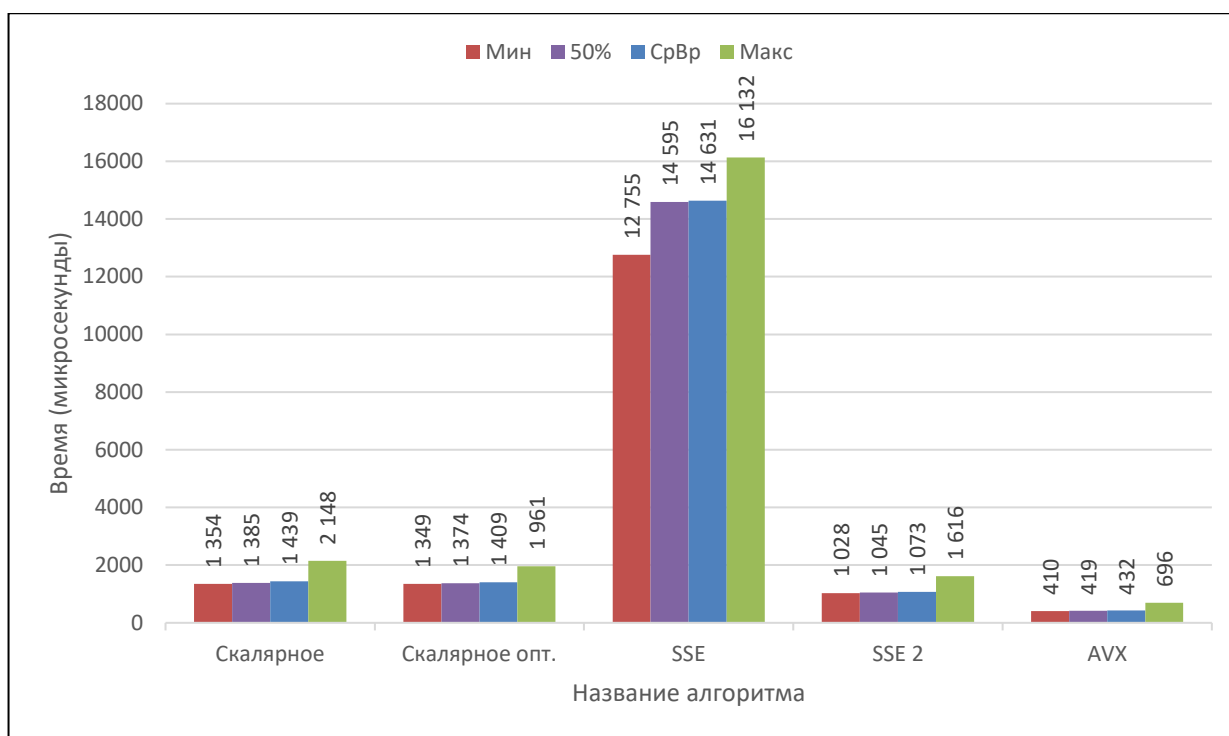


Рисунок 12 – Минимальное, медианное, среднее и максимальное время без учета наивного алгоритма

Самыми быстрыми являются AVX и SSE 2. Средними по скорости являются скалярное и скалярное оптимизированное (так как они сильно похожи). Даже используя SIMD для вычислений, можно написать код, чье время выполнения ощутимо медленнее оптимизированных вычислений на базе ALU, доказательство этому – время выполнения алгоритма SSE.

Минимальное время выполнения у SIMD алгоритмов (кроме SSE) наименьшее. У скалярного, «скалярного о.» и SSE 2 максимальное время выполнения сопоставимо друг с другом.

Коэффициент вариации – относительный показатель разброса данных, выражающий стандартное отклонение как процент от среднего значения, показывает стабильность работы алгоритма. Описание зависимости процентов вариабельности к стабильности и отклонению времени работы относительно среднего значения описаны в таблице 5 [11]. Полученные коэффициенты вариации для разработанных алгоритмов представлены на рисунке 13.

Таблица 5 – обозначения вариабельности

Процент	Вариабельность	Стабильность	Отклонение времени работы
0 - 10	Низкая	Крайне высокая	Практически незаметно
10 - 20	Средняя	Высокая	Незначительное
20 – 30	Высокая	Низкая	Сильно отклоняется от среднего
> 30	Очень высокая	Крайне низкая	Огромное, имеет высокую долю непредсказуемости

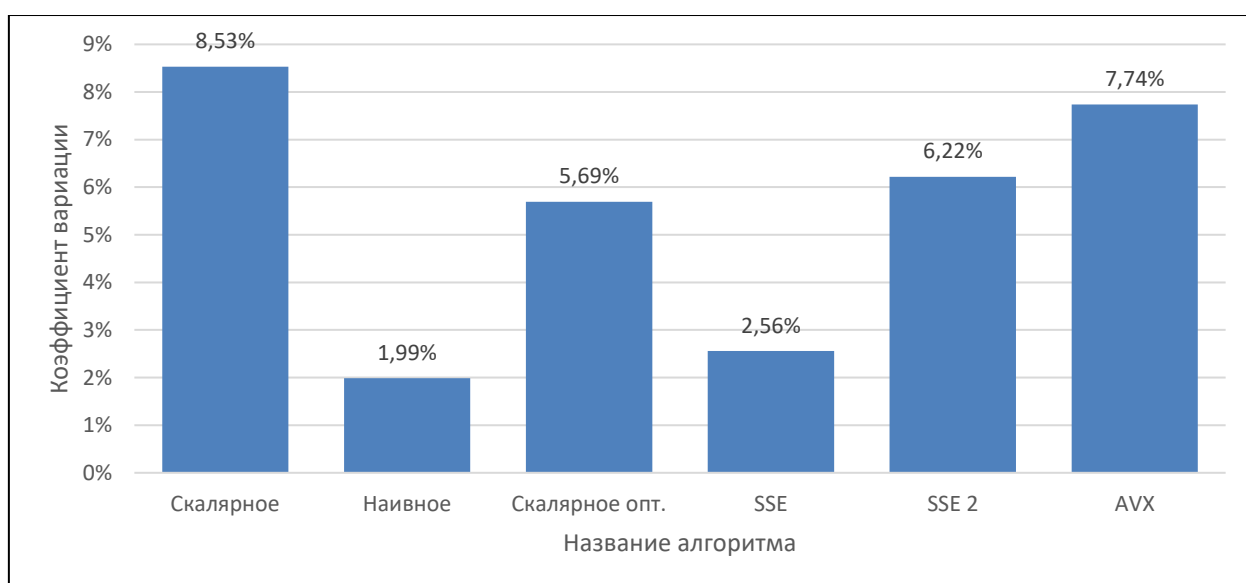


Рисунок 13 – Коэффициент вариации

Проанализировав рисунок 13, можно понять, что все алгоритмы имеют низкую вариабельность и крайне стабильны. Если сравнивать внутри этой группы, то наивный алгоритм оказывается самым стабильным (хоть и не самым быстрым), а самыми нестабильными являются скалярный, AVX и SSE 2. Достаточно стабильными являются «скалярный опт.» и SSE.

Ключевая разница между скалярными алгоритмами заключается в выделении памяти – в оптимизированном алгоритме убран лишний буфер. А так как выделение памяти не быстрый процесс, который может занять очень

много времени, уменьшение количества буферов благоприятно сказалось на коэффициенте вариации.

На рисунке 14 представлены перцентили времени выполнения: 0,1%; 1%, 50%; 99%; 99,9%; 99,99%. Краткое пояснение к обозначению перцентилей:

- 1) 0,1% - практически идеальное время выполнения;
- 2) 1% - время, которое реализация почти всегда достигает;
- 3) 50% - медиана, показывает типичное время выполнения;
- 4) 99% - показывает редкие случаи (события), которые выполняются дольше, чем обычно;
- 5) 99,9% и 99,99% - показывают очень редкие случаи, которые выполняются сильно дольше, чем обычно.

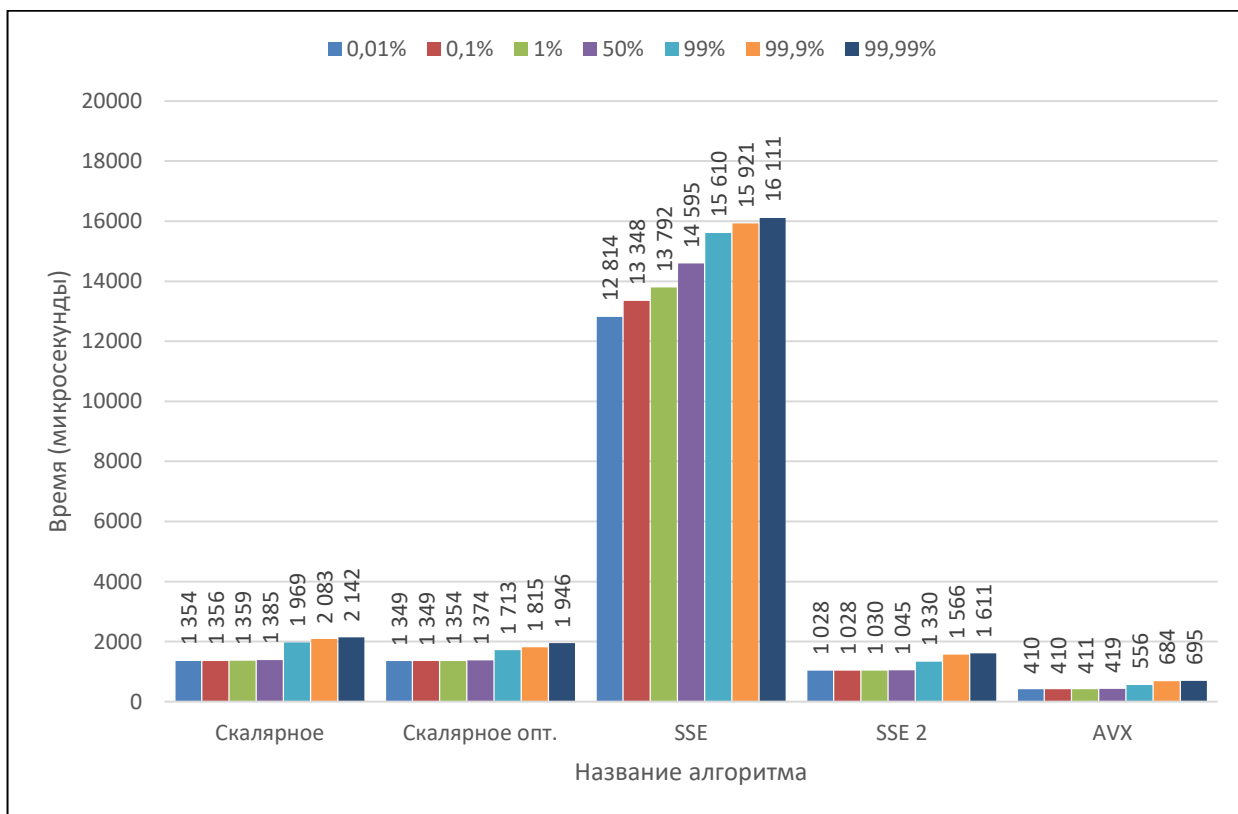


Рисунок 14 – Перцентили времени выполнения

Скалярные алгоритмы имеют практически одинаковые перцентили выполнения вплоть до 99%. После них, начинает играть роль выделение

памяти – нестабильный процесс, который плохо сказывается именно на редких событиях, так как если система искала свободный блок памяти дольше чем обычно, то это соответствующим образом скажется на наихудшем времени выполнения.

SSE 2 показывает в среднем на 31,5% лучшее время, чем скалярный оптимизированный. Также имеет хорошее время выполнения в худших случаях – на 20,8% быстрее;

AVX, как наследник SSE2 показывает результаты ещё лучше – почти в 2.5 раза быстрее (что даже быстрее предполагаемого ускорения в 2 раза за счёт увеличенных в 2 раза регистров) по медианному времени. Очень редкие события в 2.32 раза быстрее, чем у SSE2;

Все алгоритмы отличаются крайней стабильностью времени выполнения – идеальный результат почти всегда является медианой, а редкие и очень редкие события лишь незначительно больше медианы. SSE же является исключением – хоть он и является стабильным из-за коэффициента вариации, но на фоне всех остальных алгоритмов хорошо виден разброс значений. Медиана больше идеального времени выполнения почти на 13,9%, а редкие события относительно медианы на 10,3%;

При включенном параметре автовекторизации компилятора разница между SSE2 и оптимизированном скалярном алгоритме должна сильно сократиться.

Рисунок 15 показывает, во сколько раз каждая реализация быстрее наивной, по измеренным параметрам.

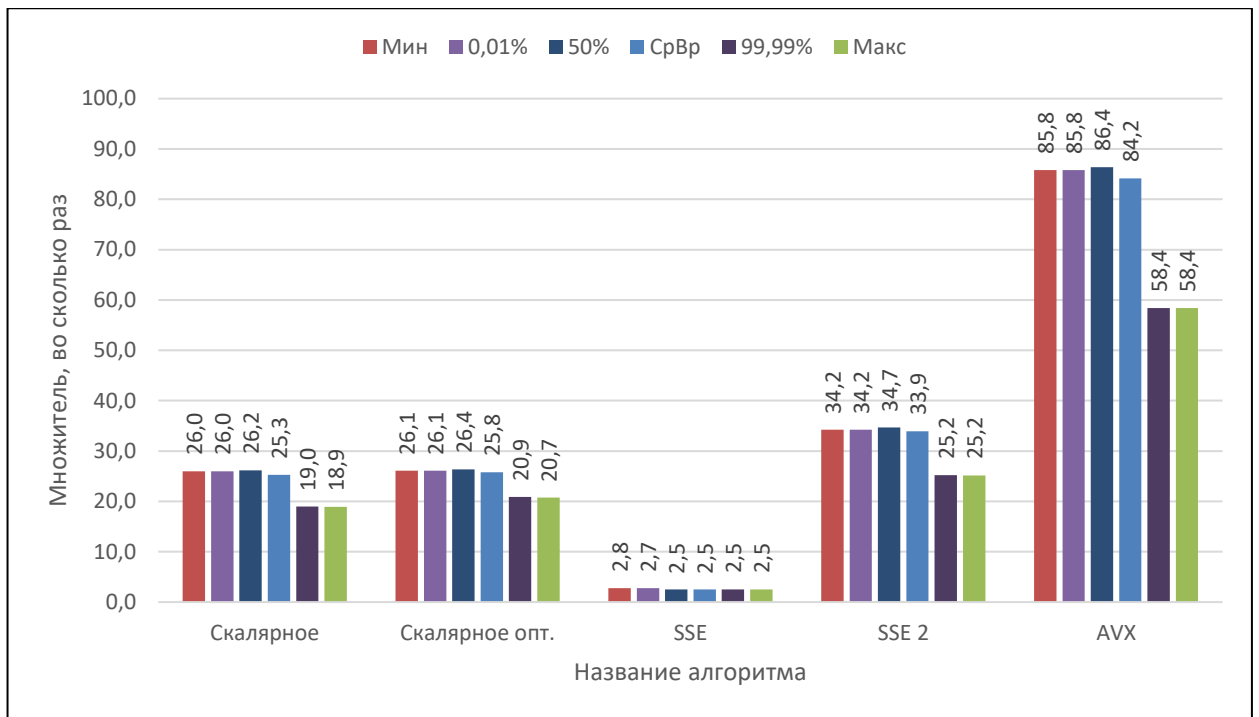


Рисунок 15 – Во сколько раз быстрее каждая реализация относительно наивной

С помощью утилиты uProf были получены дополнительные результаты. В качестве запускаемой программы был бенчмарк с параметрами: 4096*4096, 0 прогревочных итераций, 1000 итераций, 1 массив.

На рисунке 16 можно увидеть количество обращений и промахов в L1 кэш процессора, на рисунке 17 – количество обращений к L2, IPC полученных алгоритмов представлен на рисунке 18 (больше – лучше).

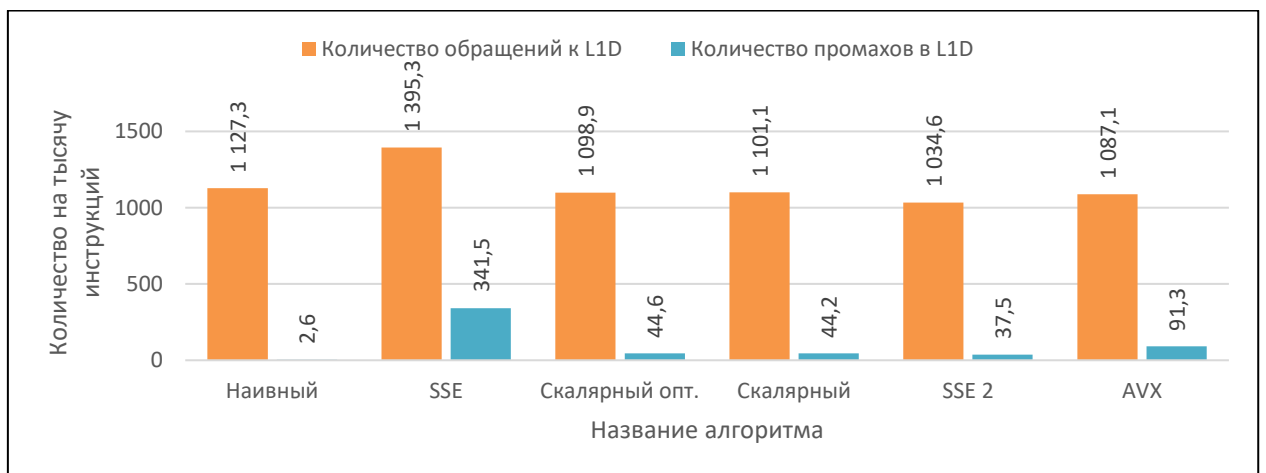


Рисунок 16 – Количество обращений к L1 кэшу процессора

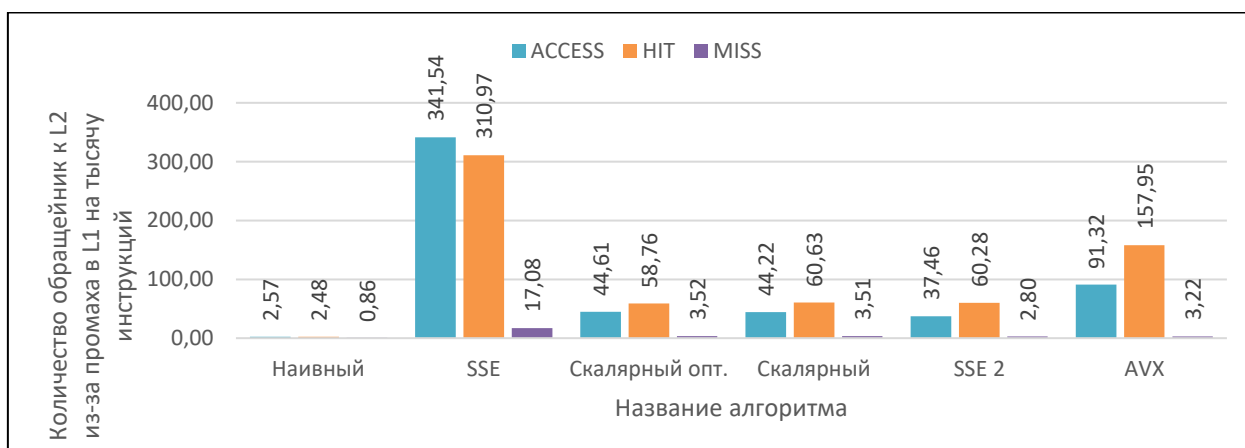


Рисунок 17 – Количество обращений к L2 кэшу процессора

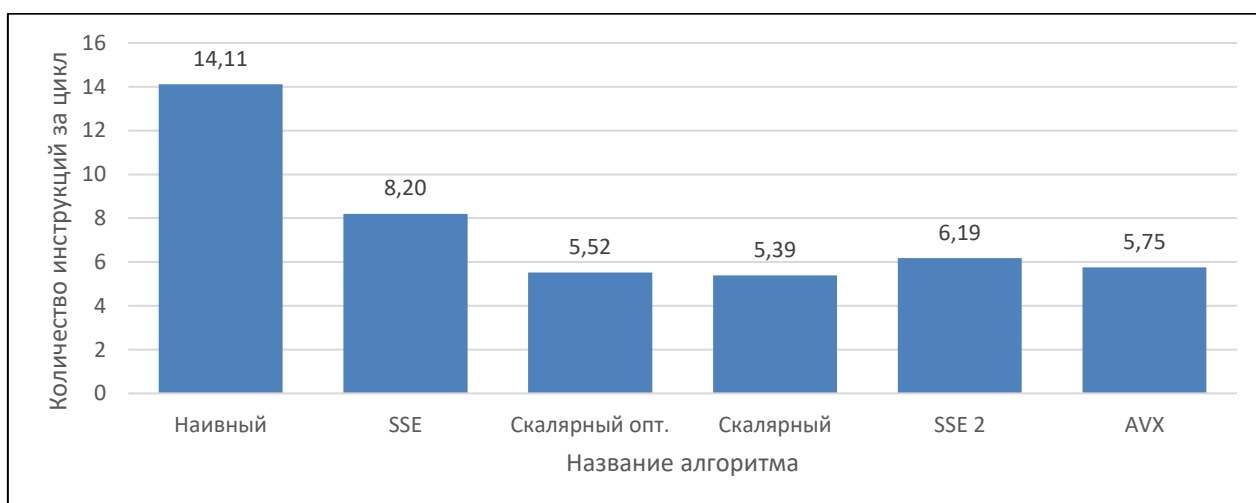


Рисунок 18 – IPC алгоритмов поиска суммы соседей

Анализ данных, представленных на рисунках 16, 17 и 18 позволяет выявить чёткую зависимость быстродействия от архитектуры и эффективности использования кэша. Для количества промахов в L1 кэш использовалась формула: $\% \text{промахов} = \text{MISS} / \text{ACCESS}$. Это обусловлено особенностями сбора данных uProf для L1 кэша. Для L2 формула иная: $\% \text{промахов} = \text{MISS} / (\text{MISS} + \text{HIT})$.

Наивная реализация имеет низкую производительность из-за высокого количества условных переходов, из-за чего предсказатель ветвлений (branch predictor) может не угадать следующий переход. Обращение к кэшам первого

и второго уровней не вносит большой разницы, наряду с большим количеством инструкций за цикл процессора, так как основной причиной низкой производительности является именно большое количество условных переходов.

Скалярные версии показывают хорошее использование кэшей: всего 4,0% промахов в L1 и 5,6% промахов в L2. Это означает, что данные очень часто находятся в кэше и их не приходится искать в далёкой L3 или очень далёкой оперативной памяти. Имеют самые низкие показатели IPC среди всех алгоритмов.

SSE имеет большие проблемы с использованием L1 – 24,5% промахов, что сильно больше чем все остальные алгоритмы. Означает, что данные в L1 находятся сильно реже. Ситуация с L2 лучше – 5,2% промахов, что ближе к скалярным версиям. Имеет большой IPC, но это не спасает алгоритм от его неэффективности.

SSE 2 хорошо работает с L1 – всего 3,6% промахов. С L2 ситуация аналогичная – 4,4%. Имеет IPC чуть больший, чем AVX, так как для обработки такого же массива данных требуется выполнить большее количество инструкций;

AVX отличается отличной работой с кэшами первого и второго уровней: 8,3% и 2% соответственно. Количество промахов в L1 из-за увеличения размера регистров велико, но это компенсируется показателем промахов в L2. Использование ILP и DLP, вместе с отличной локальностью данных позволяет данному алгоритму считаться лучшим среди разработанных в рамках данного проекта.

3.3 Измерение производительности для применения правил В и S

Используя написанный бенчмарк, были использованы следующие параметры:

- размеры массивов в элементах – $4096 * 4096$ (размер элемента 1 байт). Конечный размер массива больше объёма L1 и L2 кэша;
- количество массивов – 10;
- выравнивание в байтах – 32;
- количество итераций бенчмарка – 1000;
- количество разогревочных итераций – 1;
- единица измерения время выполнения – наносекунды.

Далее будут представлены результаты запусков бенчмарка. Гистограммы упростят понимание различий реализаций (их время выполнения, среднее время выполнения и так далее).

На рисунке 19 и 20 представлена гистограмма минимального, медианного, среднего и максимального времени выполнения алгоритмов для применения правил В и S.

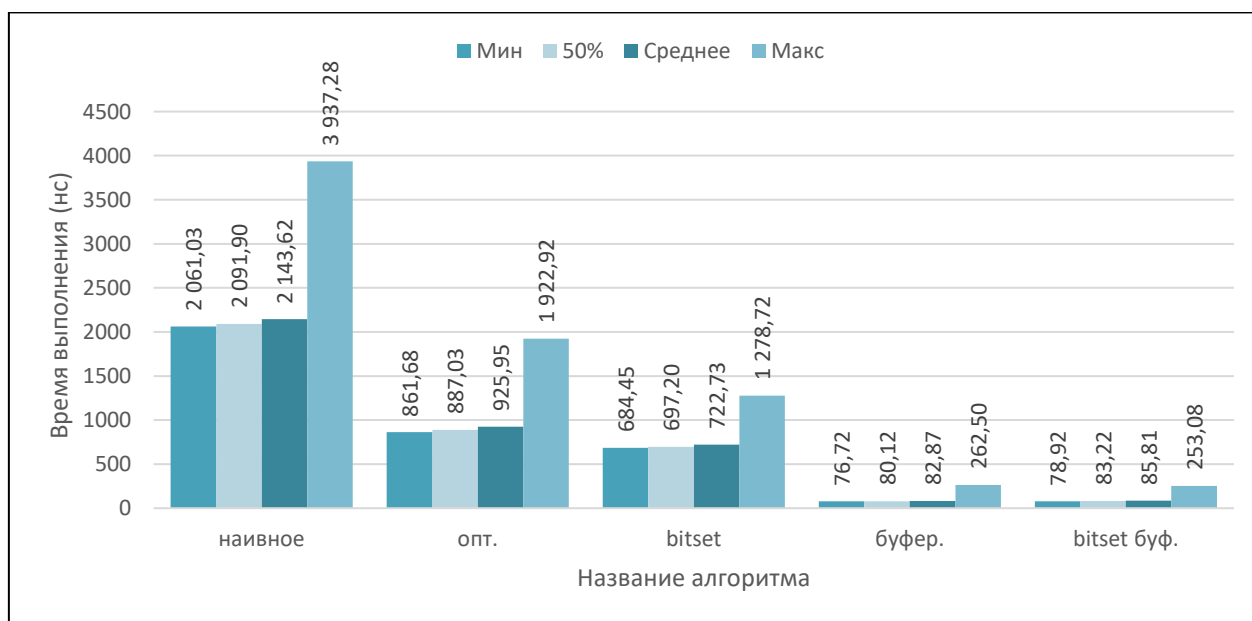


Рисунок 19 – Минимальное, медианное, среднее и максимальное время

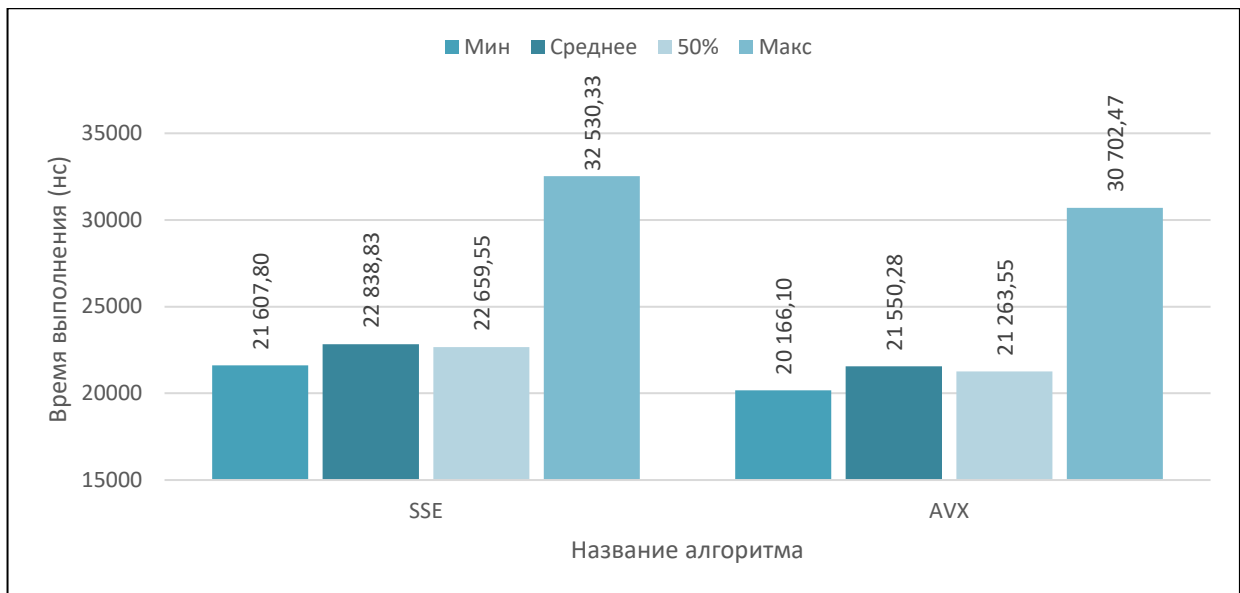


Рисунок 20 – Минимальное, медианное, среднее и максимальное время

Как видно из рисунка 20, векторизированные алгоритмы SSE и AVX отличаются худшей производительностью. В среднем их время выполнения почти в 10 раз больше, чем у наивного алгоритма.

Перцентили времени выполнения представлены на рисунках 21 и 22.

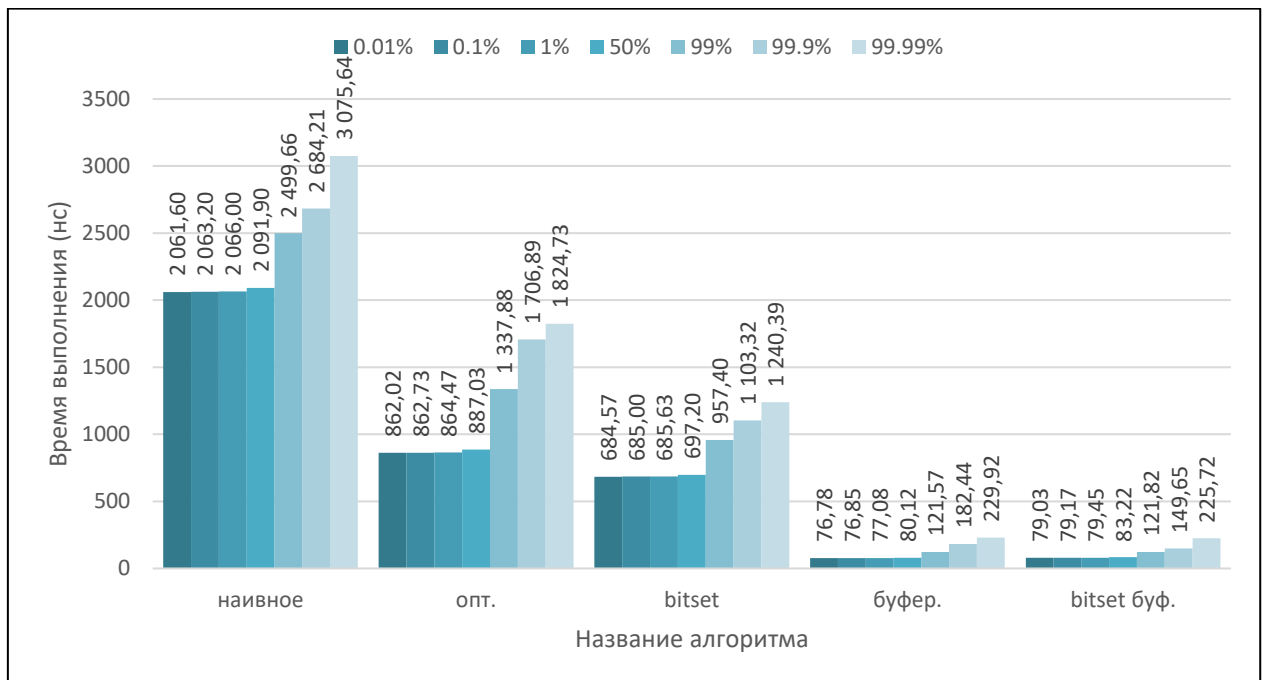


Рисунок 21 – Перцентили времени выполнения

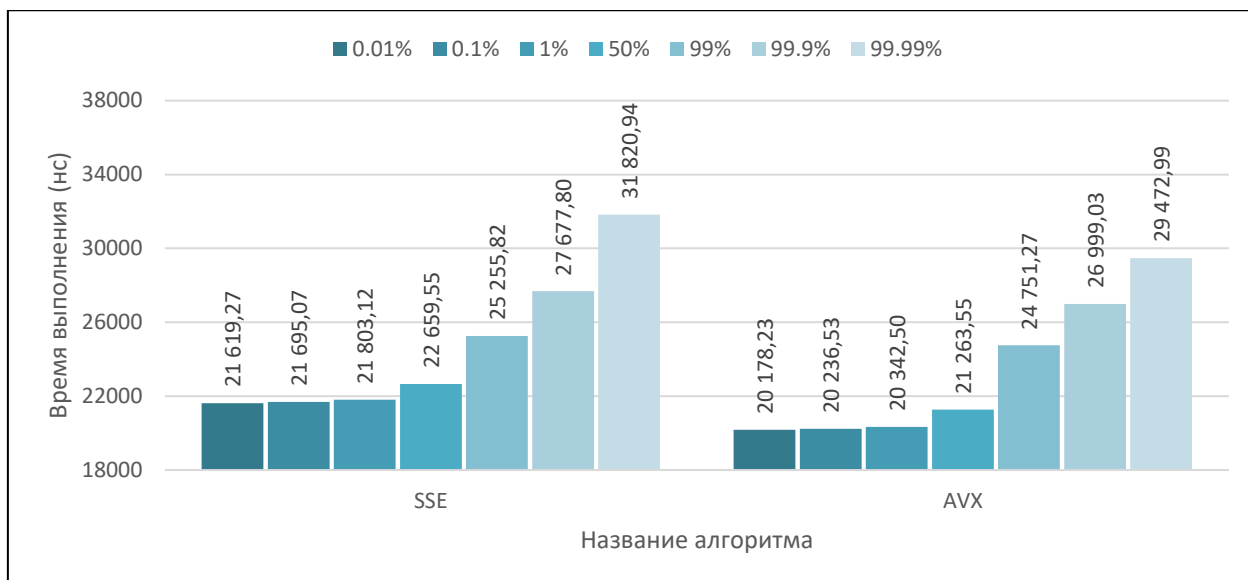


Рисунок 22 – Перцентили времени выполнения

Сравнение во сколько раз разработанные алгоритмы быстрее относительно наивного (за исключением SSE и AVX, так как они медленнее), представлены на рисунке 23.

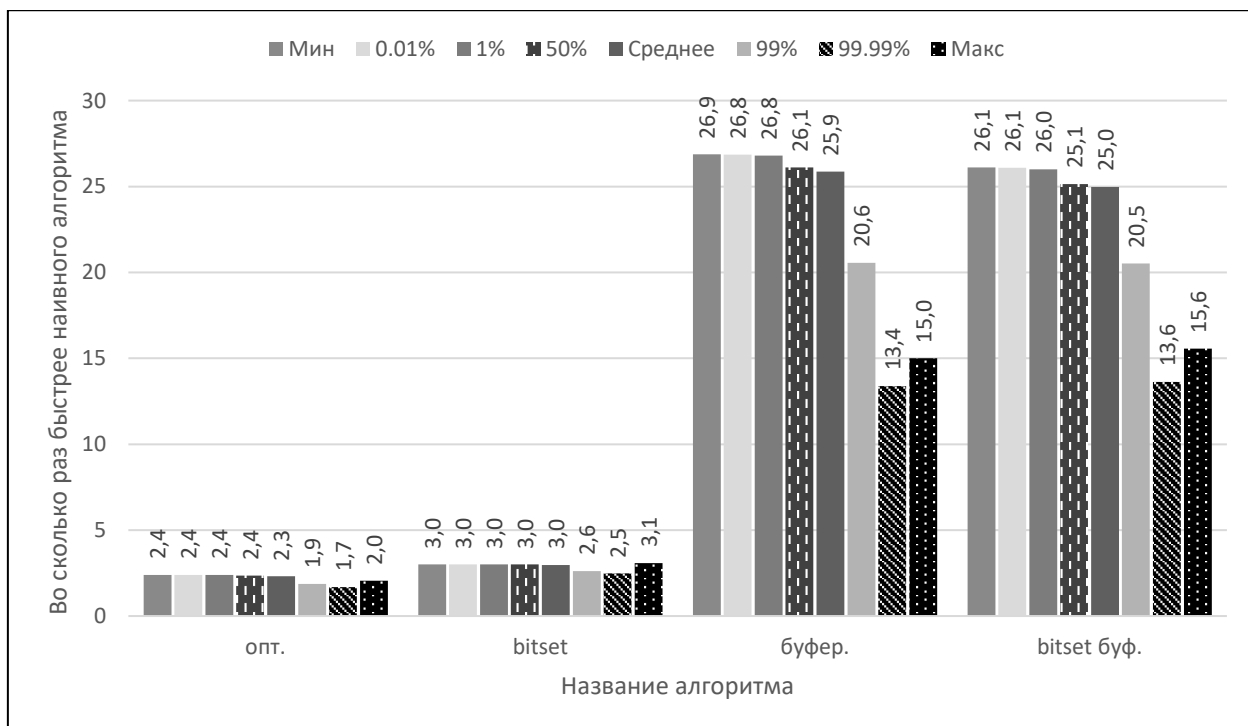


Рисунок 23 – Во сколько раз алгоритмы применения правил В и S быстрее наивного

Проанализировав рисунки 19 – 23, можно сделать следующие выводы:

- SSE и AVX имеют самое большое время работы. Это связано с тем, что на загрузку, операции сравнения и выгрузку данных уходит сильно больше тактов процессора (так как векторизированные операции требуют несколько больше циклов и на таких относительно простых задачах векторизация не приносит пользы);

- оптимизация использования блоков `if ... else` заменой на один тернарный оператор даёт ощутимый рост производительности – в среднем в 2,4 раза;

- использование `bitset` вместо `unordered_set` даст уменьшение среднего времени выполнения на 26% относительно оптимизированного;

- буферизированный и `bitset` буферизированный имеют минимальное время выполнения – в 26 раз быстрее наивного; Использование буфера равного размеру кэш-линии делённого на размер одного объекта даёт кратное ускорение – в кэш-линию процессора помещается всё окно данных, и операция сохранения в память происходит очень быстро.

В виду неудовлетворительного времени работы SSE и AVX алгоритмов, их параметры не будут замеряться с помощью утилиты AMD uProf.

Получены результаты:

- доля промахов в L1 Data Cache и L2 разработанных алгоритмов представлена на рисунке 24;

- количество неудачно предугаданных инструкций ветвления на рисунке 25. Из-за особенностей измерения uProf `bitset` равняется нулю, реальное значение находится между 0 и 25000. Вертикальная ось представлена в логарифмической шкале;

- количество инструкций за цикл можно увидеть на рисунке 26.

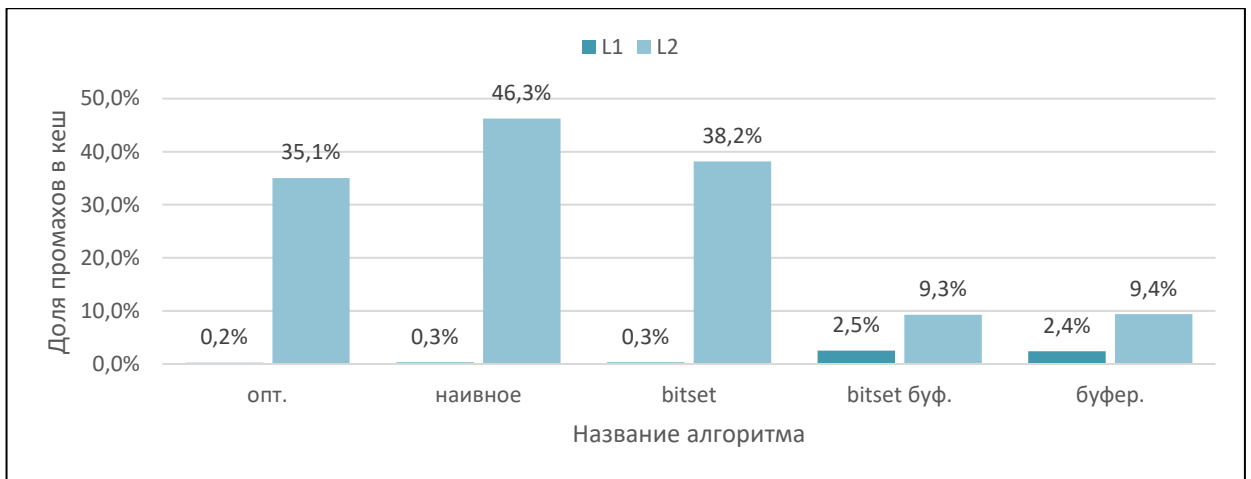


Рисунок 24 – Доля промахов в L1D и L2 алгоритмов применения правил В и S

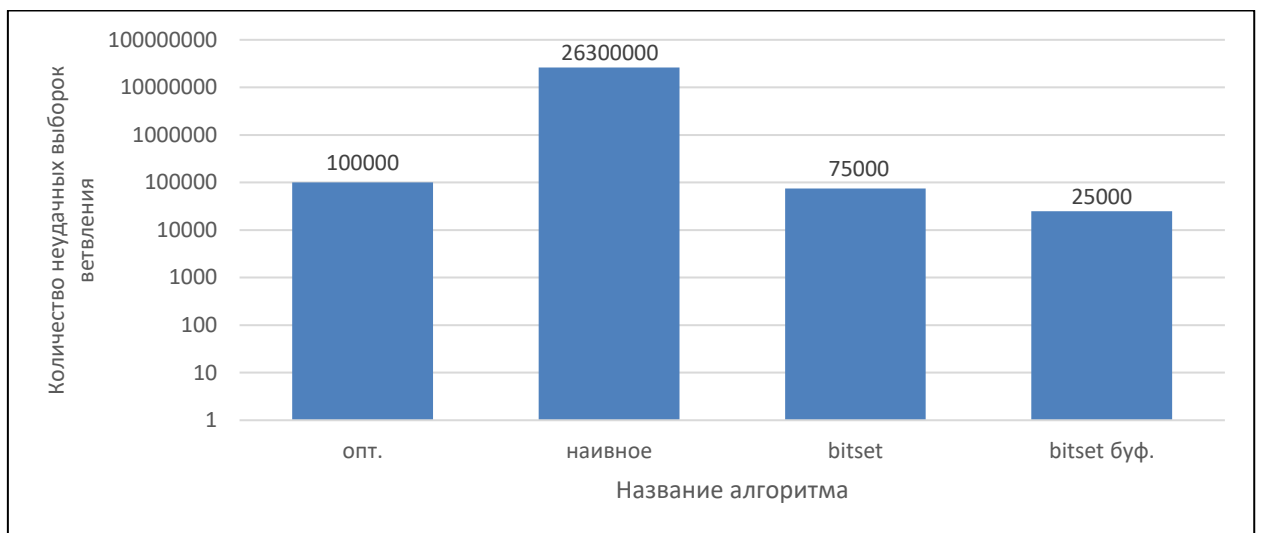


Рисунок 25 – Количество неудачных предугадываний инструкций ветвления

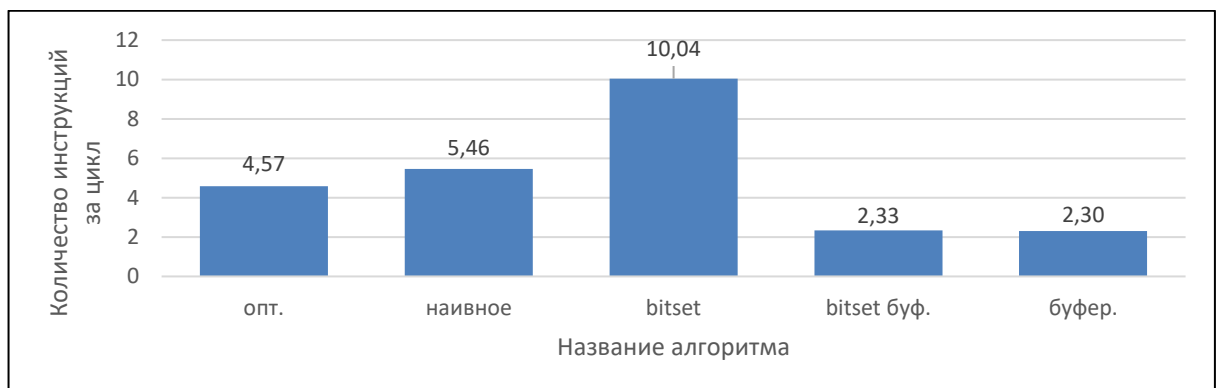


Рисунок 26 – IPC алгоритмов применения правил В и S

Согласно рисункам 24 – 26 можно сделать следующие выводы:

— наивный алгоритм очень часто ошибается (вернее branch predictor) с выбором ветки ветвления и выбирает неправильный, что вызывает простои конвейера процессора. Данные в L1 кэше находятся часто, а вот в L2 довольно редко (43% промахов). В купе с не самым большим количеством IPC имеет не самое высокое быстродействие;

— оптимизированный алгоритм имеет значительно меньшей ветвлений, из-за чего значительно реже ошибается в инструкциях ветвления. Переход от трёх if'ов к одному тернарному оператору уменьшило количество ошибочно угаданных ветвлений в 263 раза. Также это уменьшило промахи в L2 кэш на 23% относительно наивного. Из-за уменьшенного общего количества инструкций, IPC немного упал, но это не помешало алгоритму выполняться в среднем в 2,4 раза быстрее;

— благодаря особенностям работы bitset (массив упакованных bool) и константному времени доступа за $O(1)$, алгоритм на его основе предлагает максимальное быстродействие при одиночной вставке результатов. Данные в L1 находятся почти всегда, а вот в L2 часто нет – в 38 случаев из 100 происходит промах. Максимальный IPC не приносит такого же быстродействия, что и буферизация;

— буферизированные алгоритмы выделяются своей быстротой работы и маленьким количеством промахов в L1 и L2 кэш. Именно буферизация позволяет получить качественный прирост производительности – если все необходимые данные лежат в одной кэш-линии процессора, их легче скопировать в ОЗУ. Дополнительно это уменьшает саму нагрузку на память – уменьшается постоянное единичное обращение к ней (а так как ОЗУ далёкая память, то постоянное обращение несёт на себе задержки). Низкий IPC никак не влияет на производительность. Отдельно стоит отметить bitset буферизированный. В некоторых сценариях он будет чуть быстрее просто буферизированного алгоритма (в данном случае сценарий не подошёл, на

других размерах массива или другом количестве итераций разница была бы значительнее).

3.4 Сравнение общего времени выполнения

Разработанные алгоритмы являются лишь декомпозированными задачами в цикле итерации клеточного автомата. Для использования в клеточном автомате нужно объединить части с поиском суммы соседей с применением правил В и S в общие алгоритмы. Выделю следующие общие алгоритмы:

- 1) базовый – использует поиск суммы соседей и применение правил на основе наивных алгоритмов;
- 2) AVX+bitset– использует самые оптимизированные алгоритмы. Для поиска суммы соседей AVX, а для применения правил – bitset buffered;
- 3) только AVX – использует оба алгоритма AVX;
- 4) только скалярный – использует только скалярные алгоритмы (оптимизированный скалярный и bitset buf.).

Был использован бенчмарк, замерялось время выполнения 40 итераций по 2 массивам размерами $10240 * 10240$. Количество разогревочных итераций – 0. Единица времени замера - микросекунды. Самым нужным параметром является среднее и медианное время выполнения, они показаны в таблице 6, также была разработана гистограмма, которая представлена на рисунке 27.

Таблица 6 – Среднее и медианное время общих алгоритмов

Алгоритм	Среднее время (мкс)	Медианное время (мкс)
Базовый	831957,961	832261,175
AVX+bitset	153136,180	153113,475
Только AVX	9580,358	9441,650
Только скалярный	166728,140	167383,625

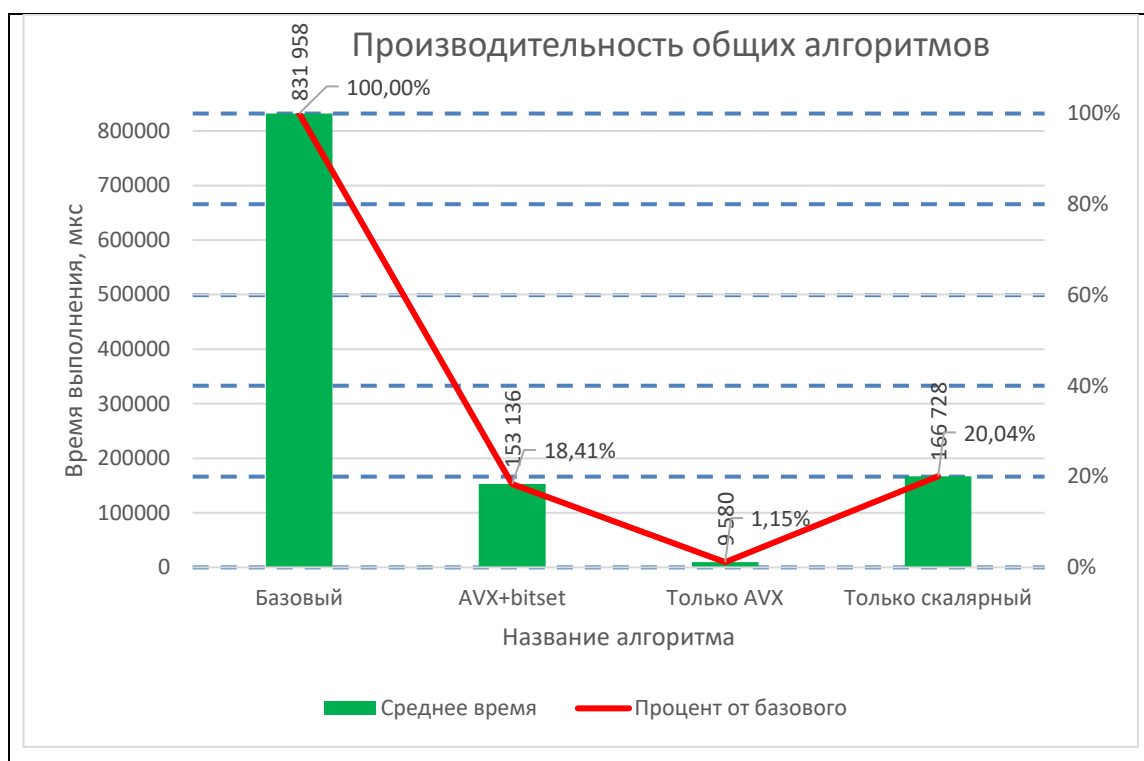


Рисунок 27 – Процентное сравнение времени выполнения общих алгоритмов

Если принять время работы базового алгоритма за 100%, то AVX+bitset будет составлять 18,40%, AVX – 1,15%, скалярный – 20,04%. Среднее время очень мало отличается от медианного, что должно положительно сказаться на времени выполнения.

Использование алгоритма AVX для поиска сумм соседей даёт значительный рост производительности, но из-за необходимости перемещения данных между регистрами AVX и стеком процессора, использование скалярного алгоритма для применения правил упирается в пропускную способность портов загрузки/выгрузки (Load/Store Units) и задержки конкретных инструкций.

Использование только AVX даёт ускорение почти в два порядка (в 86,84 раза). При использовании AVX для применения правил B и S имеет главное преимущество – данные остаются в AVX регистрах и отсутствует необходимость в их перемещении в стек или L1 кэш. Бенчмарк показал плохие результаты для AVX из-за необходимости загрузки данных из памяти. В данном алгоритме данные уже загружены в регистры для поиска сумм соседей,

а так как операция загрузки в память долгая (7 циклов согласно Intel Intrinsics Guide), этот этап минует для применения правил. Это позволяет исключить самый тяжелый этап, что и даёт колоссальную прибавку к скорости выполнения кода.

Только скалярный показывает сравнимую производительность относительно связки AVX+bitset. Данный алгоритм сковывается обращением к далёкой ОЗУ для сохранения суммы соседей и дальнейшей работы для применения правил. Для алгоритма, который не используют векторизированные операции, производительность является отличной.

Если рассмотреть время выполнения 1 итерации как количество кадров в секунду (FPS), можно выделить следующие особенности:

- базовый алгоритм выполняется за 20,79 мс, что равняется 48,1 FPS;
- AVX+bitset выполняется за 3,82 мс, что равняется 261 FPS;
- только AVX выполняется за 0,2395 мс, что равняется 41753 FPS;
- только скалярный выполнится за 4,16 мс, что равняется 240 FPS.

Дополнительно было замерено время выполнения на различных конфигурациях. Так как замеры проводились на ноутбуках с сильно различными параметрами тактовой частоты процессора и оперативной памяти, значения имеют «плавающий» характер и могут лишь использоваться для примерной оценки прироста производительности. Полученные данные представлены в таблице 7.

Таблица 7 – Результаты замера производительности на ноутбуках

Процессор	Среднее время (мкс)				Медианное время (мкс)			
	Базовый	AVX + bitset	Только AVX	Только скалярн ый	Базовый	AVX + bitset	Только AVX	Только скалярн ый
Ryzen 7 6800H	147875	32550	2632	32885	145758	32293	2596	32493

Продолжение таблицы 7

Процессор	Среднее время (мкс)				Медианное время (мкс)			
	Базовый	AVX + bitset	Только AVX	Только скалярный	Базовый	AVX + bitset	Только AVX	Только скалярный
Ryzen 7 7435HS	119312	25124	1901	28034	118300	25141	1837	28068
Core i7-13650HX	117771	33700	1863	38893	117431	33192	1878	38502

Замеры проводились по упрощённым параметрам – массив 4096*4096, 10 итераций по 1 массиву. Сравнение полученных значений будет проводиться внутри группы по процессору. Использование только AVX перед базовой реализацией даёт прирост в 56,18; 62,76 и 63,21 раз соответственно, что примерно соответствует приросту на настольном процессоре с куда большим размером L3 кэша. Использование только скалярной реализации перед базовой приносит ускорение в 4,49; 4,25; 3,03 раз соответственно, что так же примерно соответствует ускорению для текущего тестового стенда. Медианное время также очень близко к среднему, что подтверждает повторяемость увеличения производительности библиотеки.

3.5 Руководство оперативного пользователя

Алгоритм работы с динамической библиотекой:

- 1) Загрузить библиотеку, согласно соглашению вызовов cdecl;
- 2) Вызвать функцию NewResult(). Она вернёт указатель для последующих результатов, его требуется сохранить на своей стороне;
- 3) Вызвать функцию NewGameOfLife(unsigned long long width, unsigned long long height, Result* result), в качестве аргумента result передать указатель на результат из пункта 2;

- 4) Вызвать функцию TickGameOfLife(Result* result, unsigned long long tickCount), в качестве аргумента передать указатель из пункта 3. Функция вернёт код ошибки (0 – нет ошибки) и обновит значение указателя result;
- 5) Обработать полученные данные по указателю на результат из пункта 4;
- 6) Удалить выделенную память функцией DeleteGameOfLife(Result* result).

Выводы по разделу

Было проведено тестирование алгоритмов поиска суммы, применения правил, а также общих алгоритмов. На основе полученных данных и метрик uProf был проведён анализ, который объясняет разницу в производительности разработанных алгоритмов. Полученные данные позволяют подтвердить эффективность перехода на SIMD алгоритмы, так как они продемонстрировали качественный прирост производительности на фоне скалярных реализаций. Удалось уменьшить среднее время выполнения итерации в 86 раз для поля размерами 4096 на 4096 клеток.

ЗАКЛЮЧЕНИЕ

Целью проекта было создание высокопроизводительной библиотеки для моделирования клеточных автоматов на языке C++. Цель достигнута, а для этого были решены следующие поставленные задачи:

- разработана многоуровневая архитектура (ABI, Middleware, бэкенды), которая отделяет интерфейс взаимодействия от внутренней логики, а также обеспечивает выбор оптимальной аппаратной реализации с минимальными накладными расходами внутри циклов вычислений;

- спроектирована и реализована эффективная структура данных Flat2DArray, которая реализует одномерный выровненный массив. Структура улучшает локальность данных в массиве и добавляет поддержку SIMD-операций;

- реализовано ядро симуляции с оптимизированными алгоритмами подсчета соседей и применения правил (B/S), что включает специальные векторизированные версии, которые значительно производительнее (в 86 раз быстрее базовой версии);

- обеспечено межъязыковое взаимодействие через стабильный C-совместимый ABI с передачей данных по указателям, который обеспечивает семантику Zero-Copy и исключает лишние копирования.

Таким образом, разработанная библиотека подтверждает свою эффективность и доказывает высокую работоспособность предложенных подходов к разработке высокопроизводительного ПО.

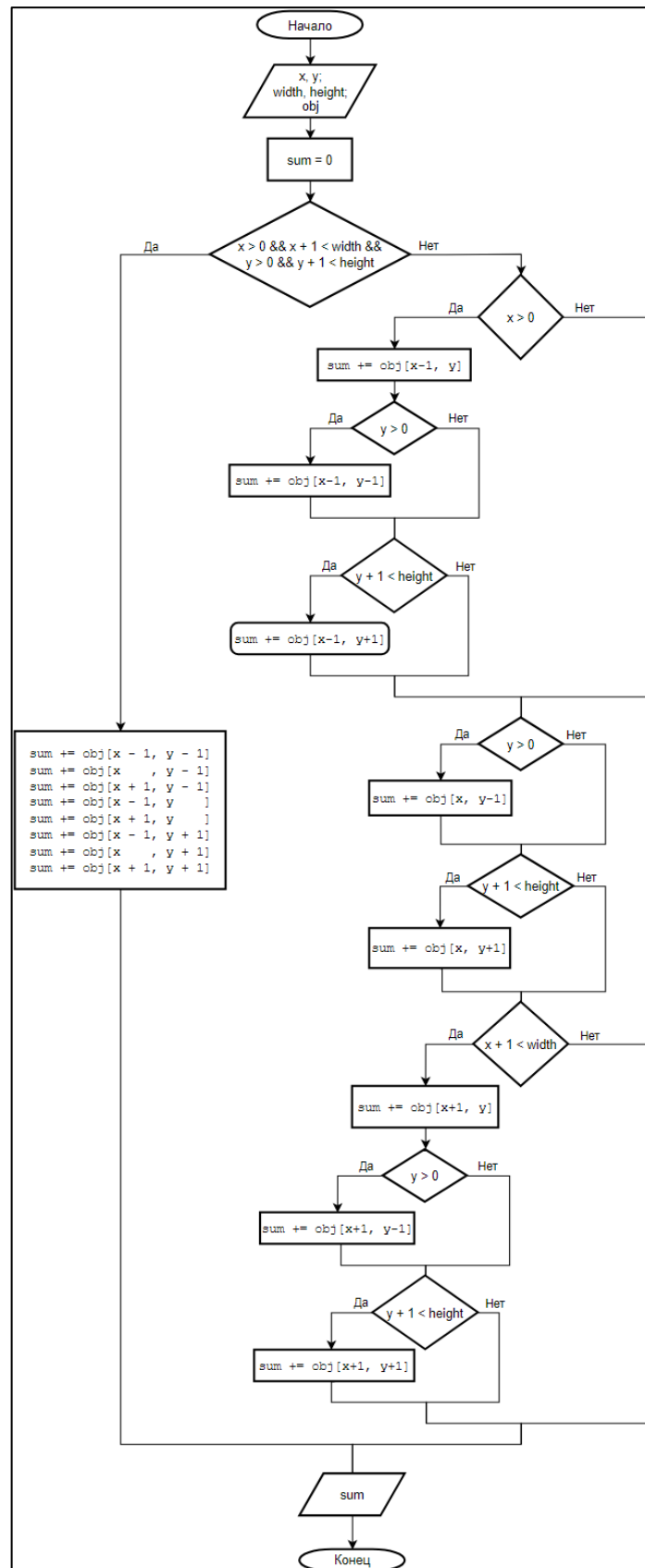
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Библиотеки в программировании: для чего нужны и какими бывают // Skillbox URL: https://skillbox.ru/media/code/biblioteki_v_programmirovanii/ (дата обращения: 10.05.2026).
2. Библиотеки в программировании: для чего нужны и какими бывают // gitverse URL: <https://gitverse.ru/blog/articles/development/135-biblioteki-v-programmirovanii-dlya-chego-nuzhny-i-kakimi-byvayut> (дата обращения: 10.05.2026).
3. Генерация Пещер // Хабр URL: <https://habr.com/en/articles/741564/> (дата обращения: 10.05.2026).
4. Игра Жизнь // Progr@m4you URL: <https://programforyou.ru/projects/life> (дата обращения: 10.05.2026).
5. Анатомия performance-critical C++ кода на примере ECS // Хабр URL: <https://habr.com/ru/articles/977462/> (дата обращения: 10.05.2026).
6. SISD архитектура: принципы работы и базовые концепты для новичков // SKY.PRO URL: <https://sky.pro/wiki/javascript/sisd-arhitektura-principy-raboty-i-osnovy-dlya-novichkov/> (дата обращения: 10.05.2026).
7. Intel® Intrinsics Guide // Intel URL: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html> (дата обращения: 10.05.2026).
8. CRTP в C++ // Хабр URL: <https://habr.com/ru/companies/otus/articles/803601/> (дата обращения: 10.05.2026).
9. Какой предел у предсказателя ветвлений? Проверили на x86 и M1 // Хабр URL: <https://habr.com/ru/companies/selectel/articles/557410/> (дата обращения: 10.05.2026).
10. What is the difference between loadu and load? // Stack Overflow URL: <https://stackoverflow.com/questions/15964367/what-is-the-difference-between-loadu-and-load> (дата обращения: 10.05.2026).

11. Расчет коэффициента вариации в Excel: формула и практическое применение // skypro URL: <https://sky.pro/wiki/analytics/raschet-koeffitsienta-variatsii-v-excel-formula-i-prakticheskoe-primenenie/> (дата обращения: 10.05.2026).

ПРИЛОЖЕНИЕ А

Алгоритм поиска суммы клетки по окрестности Мура



ПРИЛОЖЕНИЕ Б

Реализация модулей применения алгоритмов нахождения суммы по соседству Мура

Листинг 1 – Реализация модуля наивного алгоритма

```
export module normalsumv2;
export class NormalWOAllocSum : public SumRealizationBase<NormalWOAllocSum> {
public:
    NormalWOAllocSum() { name = "v lob"; }
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void test_runImpl(Flat2DArray<T>& object, Flat2DArray<T>& to_save) const
noexcept {
    const size_t width = object.width();
    const size_t height = object.height();
    T sum = 0;
    for (size_t y = 0; y < height; ++y) {
        for (size_t x = 0; x < width; ++x) {sum = 0;
            if (x > 0 && x + 1 < width && y > 0 && y + 1 < height) {
                sum += object.at(x - 1, y - 1);
                sum += object.at(x, y - 1);
                sum += object.at(x + 1, y - 1);
                sum += object.at(x - 1, y);
                sum += object.at(x + 1, y);
                sum += object.at(x - 1, y + 1);
                sum += object.at(x, y + 1);
                sum += object.at(x + 1, y + 1);
            }
            else {
                if (x > 0) {
                    sum += object.at(x - 1, y);
                    if (y > 0) sum += object.at(x - 1, y - 1);
                    if (y + 1 < height) sum += object.at(x - 1, y + 1);
                }

                if (y > 0) sum += object.at(x, y - 1);
                if (y + 1 < height) sum += object.at(x, y + 1);

                if (x + 1 < width) {
                    sum += object.at(x + 1, y);
                    if (y > 0) sum += object.at(x + 1, y - 1);
                    if (y + 1 < height) sum += object.at(x + 1, y + 1);
                }
            }
            to_save.at(x, y) = sum;
        }
    }
};
```

Листинг 2 – Реализация модуля базового скалярного алгоритма

```
export module normalsum;
export class NormalSum : public SumRealizationBase<NormalSum> {
public:
    NormalSum() { name = "normal"; }
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void test_runImpl(Flat2DArray<T>& object, Flat2DArray<T>& to_save) const
noexcept {
    const size_t width = object.width();
    const size_t height = object.height();
    T* temp_res_ptr = nullptr;
    for (short q = 0; q < height; q++) { offsets[q] = q * width;}
    T* data_ptr = object.data();
    T* res_ptr = to_save.data();
    T* ptr0 = nullptr;
    T* ptr1 = nullptr;
```

Продолжение листинга 2

```

        T* ptr2 = nullptr;
        ptr0 = data_ptr + offsets[0];
        ptr1 = data_ptr + offsets[1];
        for (size_t x = 0; x < width; x++) {temp_res_ptr[x] = ptr0[x] + ptr1[x]; }
        res_ptr[0] = temp_res_ptr[0] + temp_res_ptr[1] - ptr0[0];
        for (size_t x = 1; x < width - 1; x++) {
            res_ptr[x] = temp_res_ptr[x - 1] + temp_res_ptr[x] + temp_res_ptr[x + 1]
- ptr0[x];
        }
        res_ptr[width - 1] = temp_res_ptr[width - 2] + temp_res_ptr[width - 1] -
ptr0[width - 1];
        for (size_t y = 0; y < height - 2; y++) {
            ptr0 = data_ptr + offsets[y + 0];
            ptr1 = data_ptr + offsets[y + 1];
            ptr2 = data_ptr + offsets[y + 2];
            const size_t offset_to_result_row = offsets[y + 1];
            for (size_t x = 0; x < width; x++) {
                temp_res_ptr[x] = ptr0[x] + ptr1[x] + ptr2[x];
            }
            res_ptr[offset_to_result_row] = temp_array_for_results[0] +
temp_array_for_results[1] - ptr1[0];
            for (size_t x = 1; x < width - 1; x++) {
                res_ptr[x + offset_to_result_row] =
temp_res_ptr[x - 1] + temp_res_ptr[x] + temp_res_ptr[x + 1] - ptr1[x];
            }
            res_ptr[offset_to_result_row + width - 1] = temp_res_ptr[width - 2] +
temp_res_ptr[width - 1] - ptr1[width - 1];
        }
#pragma region last_row
        size_t last_row_offset = offsets[height - 1];
        for (size_t x = 0; x < width; x++) {temp_res_ptr[x] = ptr1[x] + ptr2[x]; }
        res_ptr[last_row_offset] = temp_res_ptr[0] + temp_res_ptr[1] - ptr2[0];
        for (size_t x = 1; x < width - 1; x++) {
            res_ptr[last_row_offset + x] = temp_res_ptr[x - 1] + temp_res_ptr[x] +
temp_res_ptr[x + 1] - ptr2[x];
        }
        res_ptr[last_row_offset + width - 1] = temp_res_ptr[width - 1] +
temp_res_ptr[width - 2] - ptr2[width - 1];
#pragma endregion
        delete[] offsets;
        delete[] temp_array_for_results;
    }
};

```

Листинг 3 – Реализация модуля оптимизированного скалярного алгоритма

```

export module iter;
export class IterSum : public SumRealizationBase<IterSum> {
public:
    IterSum() { name = "iter"; }
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void test_runImpl(Flat2DArray<T>& object, Flat2DArray<T>& to_save) const
noexcept {
        const size_t width = object.width();
        const size_t height = object.height();
        T* temp_res_ptr = nullptr;
        T* data_ptr = object.data();
        T* res_ptr = to_save.data();
        T* ptr0 = nullptr;
        T* ptr1 = nullptr;
        T* ptr2 = nullptr;
#pragma region first_lines
        ptr0 = data_ptr;
        ptr1 = data_ptr + width;
        for (size_t x = 0; x < width; x++) {temp_res_ptr[x] = ptr0[x] + ptr1[x]; }
        res_ptr[0] = temp_res_ptr[0] + temp_res_ptr[1] - ptr0[0];
        for (size_t x = 1; x < width - 1; x++) {
            res_ptr[x] = temp_res_ptr[x - 1] + temp_res_ptr[x] + temp_res_ptr[x + 1]
- ptr0[x];
        }

```

Продолжение листинга 3

```

        res_ptr[width - 1] = temp_res_ptr[width - 2] + temp_res_ptr[width - 1] -
ptr0[width - 1];
#pragma endregion
#pragma region mid
        for (size_t y = 0; y < height - 2; y++) {
            ptr0 = data_ptr + (width * y); ptr1 = data_ptr + (width * y + width);
            ptr2 = data_ptr + (width * y + 2 * width);
            const size_t offset_to_result_row = (width * y + width);
            for (size_t x = 0; x < width; x++) {
                temp_res_ptr[x] = ptr0[x] + ptr1[x] + ptr2[x];
            }
            res_ptr[offset_to_result_row] = temp_array_for_results[0] +
temp_array_for_results[1] - ptr1[0];
            for (size_t x = 1; x < width - 1; x++) {
                res_ptr[x + offset_to_result_row] =
temp_res_ptr[x - 1] + temp_res_ptr[x] + temp_res_ptr[x + 1] - ptr1[x];
            }
            res_ptr[offset_to_result_row + width - 1] = temp_res_ptr[width - 2] +
temp_res_ptr[width - 1] - ptr1[width - 1];
        }
#pragma endregion
#pragma region last_row
        size_t last_row_offset = (height - 1) * width;
        for (size_t x = 0; x < width; x++) {
            temp_res_ptr[x] = ptr1[x] + ptr2[x];
        }
        res_ptr[last_row_offset] = temp_res_ptr[0] + temp_res_ptr[1] - ptr2[0];
        for (size_t x = 1; x < width - 1; x++) {
            res_ptr[last_row_offset + x] = temp_res_ptr[x - 1] + temp_res_ptr[x] +
temp_res_ptr[x + 1] - ptr2[x];
        }
        res_ptr[last_row_offset + width - 1] = temp_res_ptr[width - 1] +
temp_res_ptr[width - 2] - ptr2[width - 1];
#pragma endregion
        delete[] temp_array_for_results;
    }
};

```

Листинг 4 – Реализация модуля SSE алгоритма

```

module;
export module sse_vertical;
export class SSEv1Sum : public SumRealizationBase<SSEv1Sum> {
public:
    SSEv1Sum() { name = "sse v1"; }
    inline const string getName_impl() const { name; }
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void test_runImpl(Flat2DArray<T>& object, Flat2DArray<T>& to_save) const
noexcept {
        constexpr size_t SSE_SIZE = 16;
        const size_t width = object.width();
        const size_t height = object.height();
        T* res_ptr = to_save.data();
        T* data_ptr = object.data();
        __m128i r0, r1, r2;
        size_t TOTAL_FULLY_IN_BLOCKS_COUNT = width / 14;
        size_t REMAINDER = width - TOTAL_FULLY_IN_BLOCKS_COUNT * 14;
        if (REMAINDER <= 2) TOTAL_FULLY_IN_BLOCKS_COUNT--;
        size_t GLOBAL_X_OFFSET = 0;
#pragma region left
        r0 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr));
        r1 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + width));
        r2 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + width * 2));
        calc_for_two_lines(r0, r1, 0);
        calc_three_lines(r0, r1, r2, width);
        for (size_t y = 2; y < height - 1; y++) {
            const size_t offset = width * y;
            r0 = r1;
            r1 = r2;
            r2 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + offset +
width));
            calc_three_lines(r0, r1, r2, offset);

```

Продолжение листинга 4

```

        }
        calc_for_two_lines(r2, r1, width * (height - 1));
#pragma endregion
#pragma region mid
        for (size_t mid_iterations_count = 0; mid_iterations_count <
TOTAL_FULLY_IN_BLOCKS_COUNT; GLOBAL_X_OFFSET += 14, mid_iterations_count++) {r0 =
_mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + GLOBAL_X_OFFSET));
        r1 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr +
GLOBAL_X_OFFSET + width));
        r2 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr +
GLOBAL_X_OFFSET + 2 * width));
        calc_for_two_lines_with_offset(r0, r1, GLOBAL_X_OFFSET);
        calc_three_lines_with_offset(r0, r1, r2, GLOBAL_X_OFFSET + width);
        for (size_t y = 2; y < height - 1; y++) {
            const size_t offset = GLOBAL_X_OFFSET + width * y;
            r0 = r1;
            r1 = r2;
            r2 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr +
offset + width));
            calc_three_lines_with_offset(r0, r1, r2, offset);
        }
        calc_for_two_lines_with_offset(r2, r1, GLOBAL_X_OFFSET + width * (height
- 1));
    }
#pragma endregion
#pragma region last
    if (GLOBAL_X_OFFSET >= width) return;
    r0 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + width - SSE_SIZE));
    r1 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + 2 * width - SSE_SIZE));
    r2 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + 3 * width - SSE_SIZE));
    calc_two_last_lines(r0, r1, width - SSE_SIZE);
    calc_three_last_lines(r0, r1, r2, 2 * width - SSE_SIZE);
    for (size_t y = 3; y < height; y++) {
        const size_t offset = width * y - SSE_SIZE;
        r0 = r1;
        r1 = r2;
        r2 = _mm_load_si128(reinterpret_cast<__m128i*>(data_ptr + offset +
width));
        calc_three_last_lines(r0, r1, r2, offset);
    }
    calc_two_last_lines(r2, r1, height * width - SSE_SIZE);
#pragma endregion
}
};

```

Листинг 5 – Реализация модуля SSE 2 алгоритма

```

module;
export module sse_horizontal;
export class SSEv2Sum : public SumRealizationBase<SSEv2Sum> {
public:
    SSEv2Sum() { name = "sse v2"; }
    inline const string getName_impl() const {return name; }
    inline static __m128i justSum(__m128i r0, __m128i r1, __m128i r2) noexcept {}
    inline static void justSum(__m128i* r0, __m128i r1, __m128i r2) noexcept {}
    inline static __m128i getNeighboursByVertical(__m128i r0) noexcept {}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void test_runImpl(Flat2DArray<T>& object, Flat2DArray<T>& to_save) const
    noexcept {
        constexpr size_t WINDOW_SIZE = 16; // размер окна для SSE = 16 элементов типа uint8_t
        const size_t width = object.width();
        const size_t height = object.height();
        auto res_ptr = to_save.data();
        auto obj_ptr = object.data();
        const size_t mid_index = width;
        const size_t blocks_count = mid_index / WINDOW_SIZE;
        const size_t remainder = width % WINDOW_SIZE;
        array<__m128i, 2> top {};array<__m128i, 2> mid {};array<__m128i, 2> low {};
        size_t x_offset = 0;
        const size_t total_blocks_count = blocks_count + bool(remainder);
#pragma region first_line
        T* y_ptr = obj_ptr;

```

Продолжение листинга 5

```

        top[0] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr));
        top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + WINDOW_SIZE));
        mid[0] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + width));
        mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + width + WINDOW_SIZE));
        low[0] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + 2 * width));
        low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + 2 * width + WINDOW_SIZE));
__m128i vertical2sum, vertical3sum;
        auto saved2VerticalSum = calcFirstTwoLines(&vertical2sum, 0);
        auto saved3VerticalSum = calcFirstThreeLines(&vertical3sum, width);
        x_offset += WINDOW_SIZE;
        for (size_t i = 1; i < blocks_count; i++, x_offset += WINDOW_SIZE) {
            T* ptr = obj_ptr + x_offset + WINDOW_SIZE;
            top[0] = top[1]; mid[0] = mid[1]; low[0] = low[1];
            top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
            mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
            low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
            saved2VerticalSum = calcTwoLines(saved2VerticalSum, &vertical2sum, x_offset);
            saved3VerticalSum = calcThreeLines(saved3VerticalSum, &vertical3sum, width +
x_offset);
        }
        if (remainder > 0) {
            x_offset = x_offset - WINDOW_SIZE + remainder;
            T* ptr = obj_ptr + x_offset;
            top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
            mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
            low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
            calcTwoLastNotFitLines(x_offset);
            calcThreeLastNotFitLines(x_offset + width);
        }
#pragma endregion
#pragma region mid_lines
        for (size_t y = 1, y_offset = y * width; y < height - 3; y++, y_offset += width) {
            x_offset = 0;
            y_ptr = obj_ptr + y_offset;
            top[0] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr));
            top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + WINDOW_SIZE));
            mid[0] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + width));
            mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + width +
WINDOW_SIZE));
            low[0] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + 2 * width));
            low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(y_ptr + 2 * width +
WINDOW_SIZE));
            saved3VerticalSum = calcFirstThreeLines(&vertical3sum, y_offset + width);
            x_offset += WINDOW_SIZE;
            T* ptr = nullptr;
            for (size_t i = 1; i < blocks_count; i++, x_offset += WINDOW_SIZE) {
                ptr = y_ptr + x_offset + WINDOW_SIZE;
                top[0] = top[1]; mid[0] = mid[1]; low[0] = low[1];
                top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
                mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
                low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
                saved3VerticalSum = calcThreeLines(saved3VerticalSum, &vertical3sum,
y_offset + x_offset + width);
            }
            if (remainder == 0) continue;
            x_offset = x_offset - WINDOW_SIZE + remainder;
            ptr = y_ptr + x_offset;
            top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
            mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
            low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
            calcThreeLastNotFitLines(y_offset + x_offset + width);
        }
#pragma endregion
#pragma region last_line
        size_t y_offset = width * height - 3 * width;
        y_ptr = obj_ptr + y_offset; T* ptr = y_ptr;
        low[0] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
        low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + WINDOW_SIZE));
        mid[0] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
        mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width + WINDOW_SIZE));
        top[0] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
        top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width + WINDOW_SIZE));
        saved2VerticalSum = calcFirstTwoLines(&vertical2sum, y_offset + 2 * width);
        saved3VerticalSum = calcFirstThreeLines(&vertical3sum, y_offset + width);

```

Окончание листинга 5

```

        x_offset = WINDOW_SIZE;
        for (size_t i = 1; i < blocks_count; i++, x_offset += WINDOW_SIZE) {
            ptr = obj_ptr + y_offset + x_offset + WINDOW_SIZE;
            low[0] = low[1]; mid[0] = mid[1]; top[0] = top[1];
            low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
            mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
            top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
            saved2VerticalSum = calcTwoLines(saved2VerticalSum, &vertical2sum, y_offset +
x_offset + 2 * width);
            saved3VerticalSum = calcThreeLines(saved3VerticalSum, &vertical3sum, y_offset +
x_offset + width);
        }
        if (remainder == 0) return;
        x_offset = x_offset - WINDOW_SIZE + remainder;
        ptr = obj_ptr + y_offset + x_offset;
        low[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr));
        mid[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + width));
        top[1] = _mm_load_si128(reinterpret_cast<__m128i*>(ptr + 2 * width));
        calcTwoLastNotFitLines(y_offset + x_offset + 2 * width);
        calcThreeLastNotFitLines(y_offset + x_offset + width);
#pragma endregion
    }
};

```

Листинг 6 – Реализация модуля AVX алгоритма

```

module; export module avx_horizontal;
export class AVXv1HorizontalSum : public SumRealizationBase<AVXv1HorizontalSum> {
public:
    AVXv1HorizontalSum() { name = "avx h"; }
    inline const string getName_impl() const { return name; }
    inline static __m256i justSum(__m256i r0, __m256i r1, __m256i r2) noexcept {}
    inline static void justSum(__m256i* r0, __m256i r1, __m256i r2) noexcept {}
    inline static __m256i getNeighboursByVertical(__m256i r0) noexcept {}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void test_runImpl(Flat2DArray<T>& object, Flat2DArray<T>& to_save) const
    noexcept {
        constexpr size_t WINDOW_SIZE = 32; // размер окна для AVX = 32 элементов типа uint8_t
        const size_t width = object.width();
        const size_t height = object.height();
        auto res_ptr = to_save.data();
        auto obj_ptr = object.data();
        const size_t mid_index = width;
        const size_t blocks_count = mid_index / WINDOW_SIZE;
        const size_t remainder = width % WINDOW_SIZE;
        array<__m256i, 2> top{};
        array<__m256i, 2> mid{};
        array<__m256i, 2> low{};
        size_t x_offset = 0;
        const size_t total_blocks_count = blocks_count + bool(remainder);
        T* y_ptr = obj_ptr;
        top[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr));
        top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + WINDOW_SIZE));
        mid[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + width));
        mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + width + WINDOW_SIZE));
        low[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + 2 * width));
        low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + 2 * width +
WINDOW_SIZE));
        __m256i vertical2sum{}, vertical3sum{};
        auto saved2VerticalSum = calcFirstTwoLines(&vertical2sum, 0);
        auto saved3VerticalSum = calcFirstThreeLines(&vertical3sum, width);
        x_offset += WINDOW_SIZE;
        for (size_t i = 1; i < blocks_count; i++, x_offset += WINDOW_SIZE) {
            T* ptr = obj_ptr + x_offset + WINDOW_SIZE;
            top[0] = top[1]; mid[0] = mid[1]; low[0] = low[1];
            top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
            mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
            low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width));
            saved2VerticalSum = calcTwoLines(saved2VerticalSum, &vertical2sum, x_offset);
            saved3VerticalSum = calcThreeLines(saved3VerticalSum, &vertical3sum, width +
x_offset);
        }
    }
};

```


Продолжение листинга 6

```

        if (remainder > 0) {
            x_offset = x_offset - WINDOW_SIZE + remainder;
            T* ptr = obj_ptr + x_offset;
            top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
            mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
            low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width));
            calcTwoLastNotFitLines(x_offset);
            calcThreeLastNotFitLines(x_offset + width);
        }
        for (size_t y = 1, y_offset = y * width; y < height - 3; y++, y_offset += width) {
            x_offset = 0; y_ptr = obj_ptr + y_offset;
            top[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr));
            top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + WINDOW_SIZE));
            mid[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + width));
            mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + width +
WINDOW_SIZE));

            low[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + 2 * width));
            low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(y_ptr + 2 * width +
WINDOW_SIZE));

            saved3VerticalSum = calcFirstThreeLines(&vertical3sum, y_offset + width);
            x_offset += WINDOW_SIZE; T* ptr = nullptr;
            for (size_t i = 1; i < blocks_count; i++, x_offset += WINDOW_SIZE) {
                ptr = y_ptr + x_offset + WINDOW_SIZE;
                top[0] = top[1];
                mid[0] = mid[1];
                low[0] = low[1];
                top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
                mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
                low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 *
width));
                saved3VerticalSum = calcThreeLines(saved3VerticalSum, &vertical3sum,
y_offset + x_offset + width);
            }
            if (remainder == 0) continue;
            x_offset = x_offset - WINDOW_SIZE + remainder;
            ptr = y_ptr + x_offset;
            top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
            mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
            low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width));
            calcThreeLastNotFitLines(y_offset + x_offset + width);
        }
        size_t y_offset = width * height - 3 * width; y_ptr = obj_ptr + y_offset; T* ptr = y_ptr;
        low[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
        low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + WINDOW_SIZE));
        mid[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
        mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width + WINDOW_SIZE));
        top[0] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width));
        top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width + WINDOW_SIZE));
        saved2VerticalSum = calcFirstTwoLines(&vertical2sum, y_offset + 2 * width);
        saved3VerticalSum = calcFirstThreeLines(&vertical3sum, y_offset + width);
        x_offset = WINDOW_SIZE;
        for (size_t i = 1; i < blocks_count; i++, x_offset += WINDOW_SIZE) {
            ptr = obj_ptr + y_offset + x_offset + WINDOW_SIZE;
            low[0] = low[1]; mid[0] = mid[1]; top[0] = top[1];
            low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
            mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
            top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width));
            saved2VerticalSum = calcTwoLines(saved2VerticalSum, &vertical2sum, y_offset +
x_offset + 2 * width);
            saved3VerticalSum = calcThreeLines(saved3VerticalSum, &vertical3sum, y_offset +
x_offset + width);
        }
        if (remainder == 0) return; x_offset = x_offset - WINDOW_SIZE + remainder;
        ptr = obj_ptr + y_offset + x_offset;
        low[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr));
        mid[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + width));
        top[1] = _mm256_load_si256(reinterpret_cast<__m256i*>(ptr + 2 * width));
        calcTwoLastNotFitLines(y_offset + x_offset + 2 * width);
        calcThreeLastNotFitLines(y_offset + x_offset + width);
#pragma endregion
    });

```

ПРИЛОЖЕНИЕ В

Реализация модулей применения правил В и S

Листинг 1 – Реализация модуля на основе SSE

```
module;
export module sseRule;
private:
    array<uint8_t, 32> LUT;
public:
    SseRule() {name = "sse"; LUT = MasksCreator::Generate32bitLUTByBS({3}, {2, 3}); }
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& neighbours)
const noexcept {
    constexpr size_t SSE_WINDOW = 16;

    const size_t width = object.width(), height = object.height(), capacity = width *
height;

    const size_t iter_count = capacity / SSE_WINDOW;
    const size_t is_not_fit = capacity - SSE_WINDOW * iter_count;
    T* dataPtr = object.data(); T* resPtr = neighbours.data();
    const __m128i lut_birth = _mm_load_si128((const __m128i*)LUT.data());
    const __m128i lut_survive = _mm_load_si128((const __m128i*)LUT.data() + 16);
    auto sum = [&dataPtr, &resPtr, &lut_birth, &lut_survive] (size_t load_offset) ->
__m128i {
        const __m128i one = _mm_set1_epi8(1);
        __m128i state = _mm_load_si128((const __m128i*)(dataPtr + load_offset));
        __m128i neigh = _mm_load_si128((const __m128i*)(resPtr + load_offset));
        __m128i birth_res = _mm_shuffle_epi8(lut_birth, neigh);
        __m128i surv_res = _mm_shuffle_epi8(lut_survive, neigh);
        __m128i live_mask = _mm_cmpeq_epi8(state, one);
        __m128i result = _mm_blendv_epi8(birth_res, surv_res, live_mask);
        return result;
    };

    for (size_t x = 0, i = 0; i < iter_count; x += SSE_WINDOW, i++) {
        auto res = sum(x);
        _mm_store_si128(reinterpret_cast<__m128i*>(resPtr + x), res);
    }

    if (is_not_fit) {
        size_t x = 0;
        if (capacity > SSE_WINDOW) x = capacity - SSE_WINDOW;
        auto res = sum(x);
        using std::memcpy;
        alignas(__m128i) uint8_t temp_to_remove[SSE_WINDOW] {};
        _mm_storeu_si128(reinterpret_cast<__m128i*>(temp_to_remove), res);
        memcpy(
            resPtr + (capacity - is_not_fit),
            temp_to_remove + SSE_WINDOW - is_not_fit,
            is_not_fit
        );
    }
}
};
```

Листинг 2 – Реализация модуля на основе AVX

```
module;
export module avxRule;
export class AvxRule : public SumFilterBase<AvxRule> {
private:
    array<uint8_t, 32> LUT_B; array<uint8_t, 32> LUT_S;
public:
    AvxRule() {name = "avx"; LUT_B = MasksCreator::GenerateLUT_B({ 3 });
        LUT_S = MasksCreator::GenerateLUT_S({ 2, 3 });}
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
```

Продолжение листинга 2

```

__declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& neighbours)
const noexcept {
    constexpr size_t AVX2_WINDOW = 32;
    const size_t width = object.width(), height = object.height(), capacity = width * height;
    const size_t iter_count = capacity / AVX2_WINDOW;
    const size_t is_not_fit = capacity - AVX2_WINDOW * iter_count;
    T* dataPtr = object.data();
    T* resPtr = neighbours.data();
    const __m256i lutB = _mm256_load_si256((__m256i*) (LUT_B.data()));
    const __m256i lutS = _mm256_load_si256((__m256i*) (LUT_S.data()));
    auto sum = [&dataPtr, &resPtr, &lutB, &lutS] (size_t load_offset) -> __m256i {
        __m256i state = _mm256_load_si256(reinterpret_cast<__m256i*>(dataPtr +
load_offset));
        __m256i neigh = _mm256_load_si256(reinterpret_cast<__m256i*>(resPtr +
load_offset));

        const __m256i one = _mm256_set1_epi8(1);
        __m256i res_B = _mm256_shuffle_epi8(lutB, neigh);
        __m256i res_S = _mm256_shuffle_epi8(lutS, neigh);
        __m256i live_mask = _mm256_cmpeq_epi8(state, one);
        return _mm256_blendv_epi8(res_B, res_S, live_mask);
    };
    for (size_t x = 0, i = 0; i < iter_count; x += AVX2_WINDOW, i++) {
        auto res = sum(x);
        _mm256_store_si256(reinterpret_cast<__m256i*>(resPtr + x), res);
    }
    if (is_not_fit) {
        size_t x = 0;
        if (capacity > AVX2_WINDOW) x = capacity - AVX2_WINDOW;
        auto res = sum(x);
        alignas(__m256i) uint8_t temp_to_remove[AVX2_WINDOW] {};
        _mm256_storeu_si256(reinterpret_cast<__m256i*>(temp_to_remove), res);
        memcpy(
            resPtr + (capacity - is_not_fit),
            temp_to_remove + AVX2_WINDOW - is_not_fit,
            is_not_fit
        );
    }
}
};

```

Листинг 3 – Реализация модуля на основе наивной реализации

```

export module normalRule;
export class NormalRule: public SumFilterBase<NormalRule> {
private:
    std::unordered_set<int> ruleB;
    std::unordered_set<int> ruleS;
public:
    NormalRule() {name = "normal";ruleB = { 3 };ruleS = { 2, 3 }; }
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& neighbours)
const noexcept {
        const size_t object_capacity = object.width() * object.height();
        for (size_t x = 0; x < object_capacity; x++) {
            T& neighbours_count = neighbours[x];
            bool alive = object[x];
            if (alive && ruleS.contains(neighbours_count)){neighbours_count = true;}
            else if (!alive && ruleB.contains(neighbours_count)) {
                neighbours_count = true;
            }
            else {neighbours_count = false;}
        }
    }
};

```

Листинг 4 – Реализация модуля на основе оптимизированной реализации

```
export module normalOptimizedRule;
export class NormalRuleIfOpt : public SumFilterBase<NormalRuleIfOpt> {
private:
    std::unordered_set<int> ruleB;
    std::unordered_set<int> ruleS;
public:
    NormalRuleIfOpt() {name = "norm o";ruleB = { 3 };ruleS = { 2, 3 };}
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& to_save)
const noexcept {
        const size_t object_capacity = object.width() * object.height();
        for (size_t x = 0; x < object_capacity; x++) {
            T& saved = to_save[x];
            bool alive = object[x];
            bool b_contains = ruleB.contains(saved);
            bool s_contains = ruleS.contains(saved);
            saved = alive ? s_contains : b_contains;
        }
    }
};
```

Листинг 5 – Реализация модуля с использованием bitset

```
module;
export module bitsetRule;
export class BitsetRule : public SumFilterBase<BitsetRule> {
private:
    bitset<9> ruleB;
    bitset<9> ruleS;
public:
    BitsetRule() {name = "bitset"; ruleB.set(3);ruleS.set(2).set(3);}
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& to_save)
const noexcept {
        const size_t object_capacity = object.width() * object.height();
        for (size_t x = 0; x < object_capacity; x++) {
            T& saved = to_save[x];
            bool alive = object[x];
            bool b_contains = ruleB.test(saved);
            bool s_contains = ruleS.test(saved);
            saved = alive ? s_contains : b_contains;
        }
    }
};
```

Листинг 6 – Реализация модуля с использованием буфера

```
export module bufferedRule;
export class BufferedRule : public SumFilterBase<BufferedRule> {
private:
    std::unordered_set<int> ruleB;
    std::unordered_set<int> ruleS;
    static constexpr size_t windowSize = 8;
public:
    BufferedRule() {name = "buffered";ruleB = { 3 };ruleS = { 2, 3 };}
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& neighbours)
const noexcept {
        const size_t object_capacity = object.width() * object.height();
        T* dataPtr = object.data();
        T* neighPtr = neighbours.data();
        array<uint8_t, windowSize> res {};
        size_t x = 0;
        for (; x < object_capacity - windowSize; x += windowSize) {
            for (size_t i = 0; i < windowSize; i++) {
                T& neighbours_count = neighPtr[x + i];
            }
        }
    }
};
```

Продолжение листинга 6

```

        bool alive = dataPtr[x + i];
        bool b_contains = ruleB.contains(neighbours_count);
        bool s_contains = ruleS.contains(neighbours_count);
        res[i] = alive ? s_contains : b_contains;
    }
    memcpy(neighPtr + x, res.data(), windowSize);
}
size_t remainder = object_capacity % windowSize;
if (remainder != 0) {
    for (; x < object_capacity; x++) {
        T& neighbours_count = neighPtr[x];
        bool alive = dataPtr[x];
        bool b_contains = ruleB.contains(neighbours_count);
        bool s_contains = ruleS.contains(neighbours_count);
        neighbours_count = alive ? b_contains : s_contains;
    }
}
};

```

Листинг 7 – Реализация модуля с использованием буфера для bitset

```

module;
export module bitsetBufferedRule;
export class BitsetBufferedRule : public SumFilterBase<BitsetBufferedRule> {
private:
    bitset<9> ruleB;
    bitset<9> ruleS;
    static constexpr size_t windowSize = 8;
public:
    BitsetBufferedRule() {name = "bit buf";ruleB.set(3);ruleS.set(2).set(3);}
    inline const string getName_impl() const {return name;}
    template<typename T> requires allowed_type<T>
    __declspec(noinline) void applyRule_impl(Flat2DArray<T>& object, Flat2DArray<T>& to_save)
const noexcept {
        const size_t object_capacity = object.width() * object.height();
        T* dataPtr = object.data();
        T* resPtr = to_save.data();
        array<uint8_t, windowSize> res {};
        size_t x = 0;
        for (; x < object_capacity - windowSize; x += windowSize) {
            for (size_t i = 0; i < windowSize; i++) {
                T& neighbours_count = resPtr[x + i];
                bool alive = dataPtr[x + i];
                bool s_contains = ruleS.test(neighbours_count);
                bool b_contains = ruleB.test(neighbours_count);
                res[i] = alive ? s_contains : b_contains;
            }
            memcpy(resPtr + x, res.data(), windowSize);
        }
        size_t remainder = object_capacity % windowSize;
        if (remainder != 0) {
            for (; x < object_capacity; x++) {
                T& neighbours_count = resPtr[x];
                bool alive = dataPtr[x];
                bool s_contains = ruleS.test(neighbours_count);
                bool b_contains = ruleB.test(neighbours_count);
                neighbours_count = alive ? s_contains : b_contains;
            }
        }
    }
};

```

ПРИЛОЖЕНИЕ Г

Графический материал

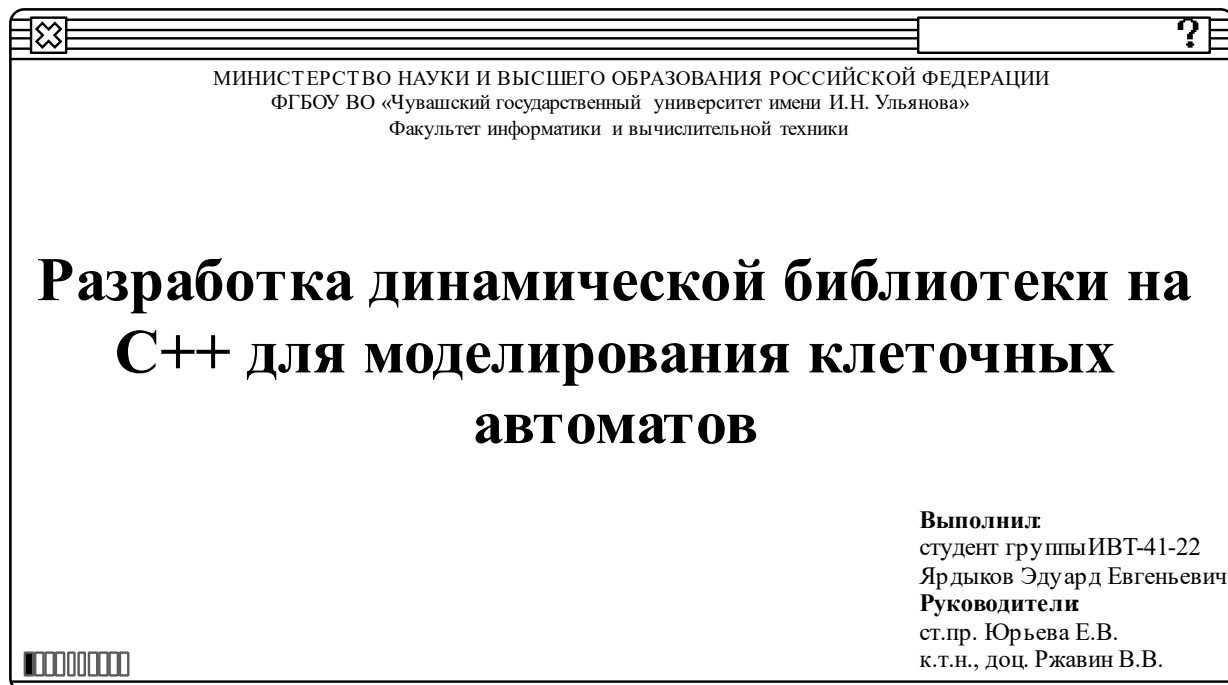


Рисунок Г.1 – Титульный лист

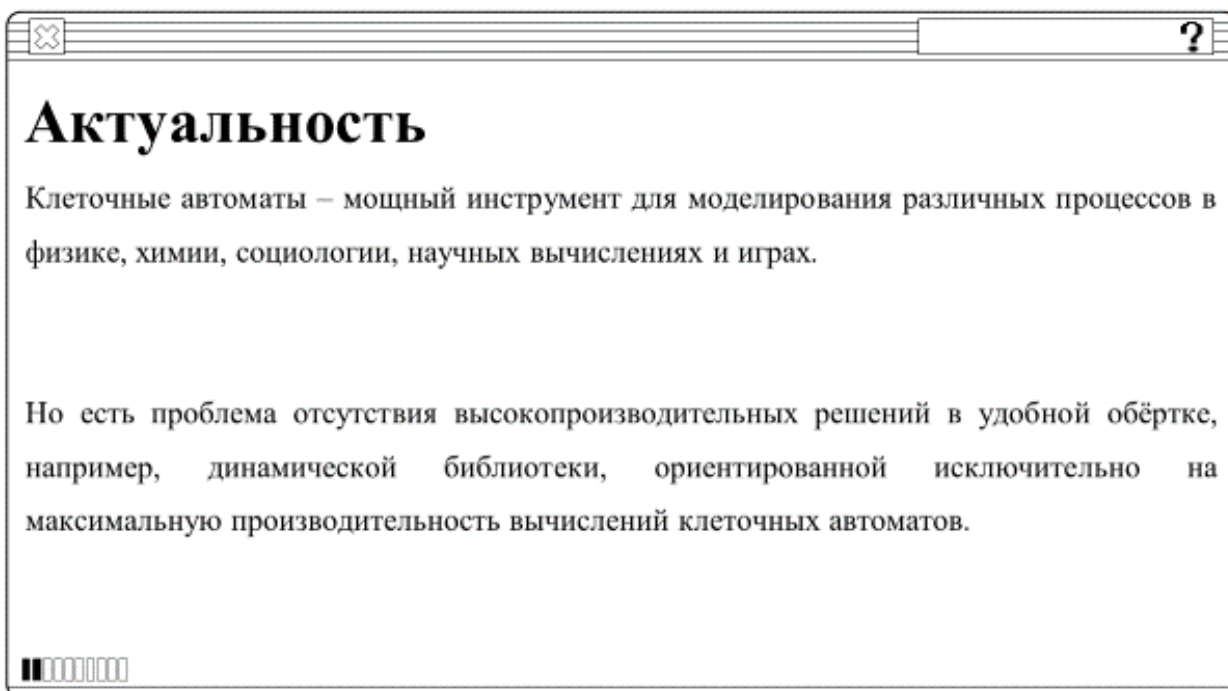


Рисунок Г.2 – Актуальность

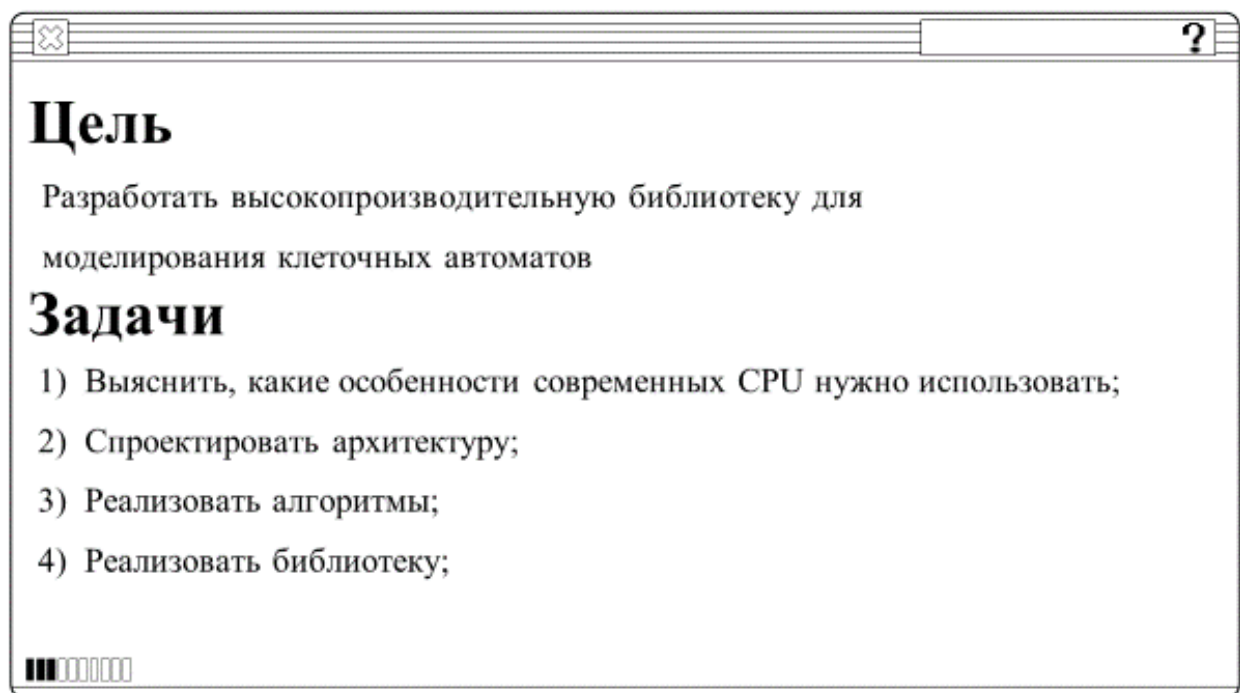


Рисунок Г.3 – Цель и задачи

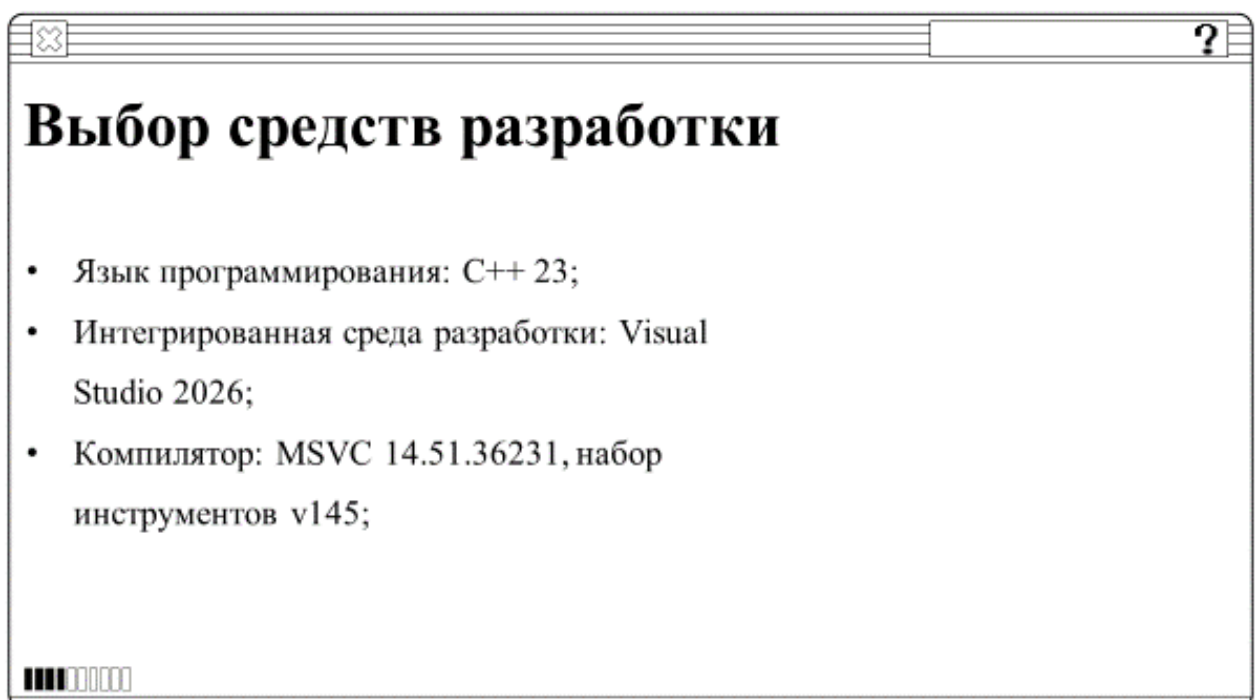


Рисунок Г.4 – Выбор средств программирования

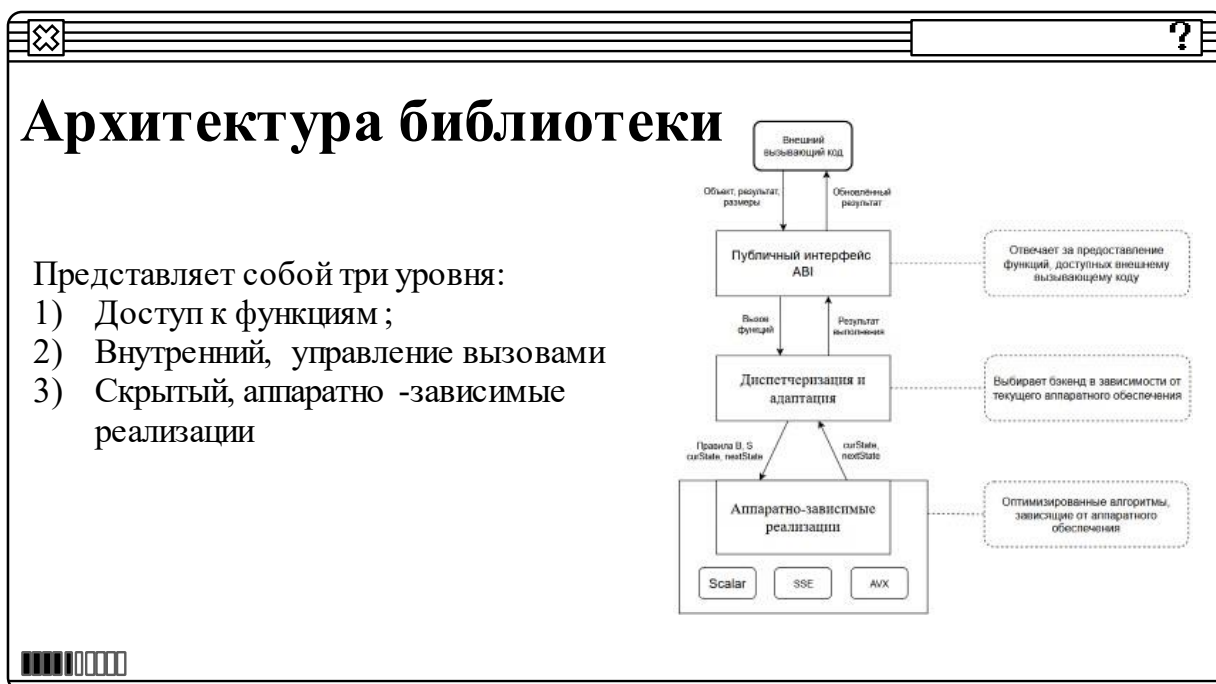


Рисунок Г.5 – Архитектура библиотеки

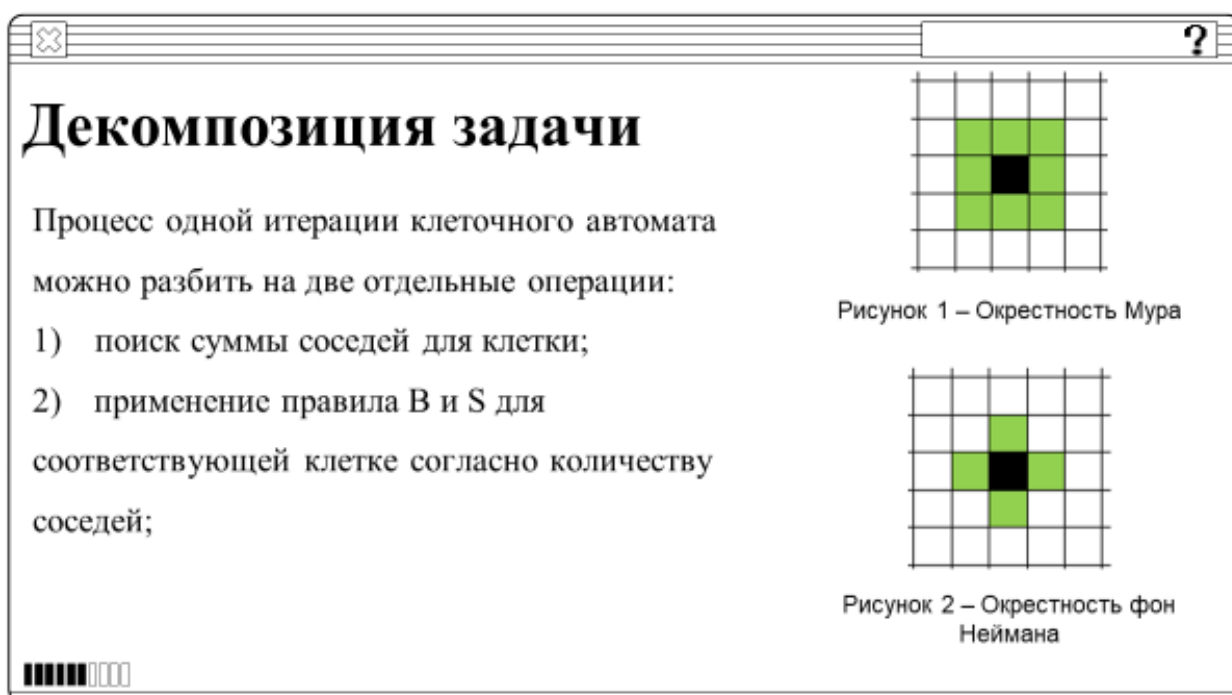


Рисунок Г.6 – Декомпозиция задачи

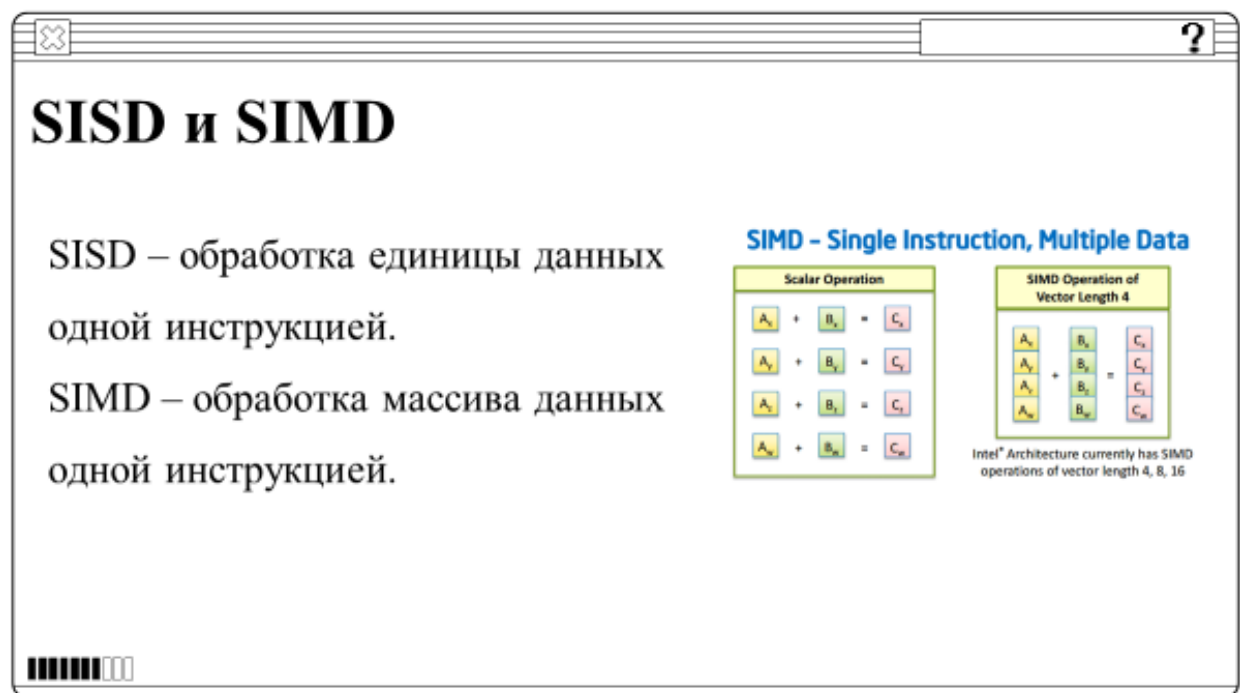


Рисунок Г.7 – SISD и SIMD



Рисунок Г.8 – Производительность разработанных алгоритмов поиска сумм соседей

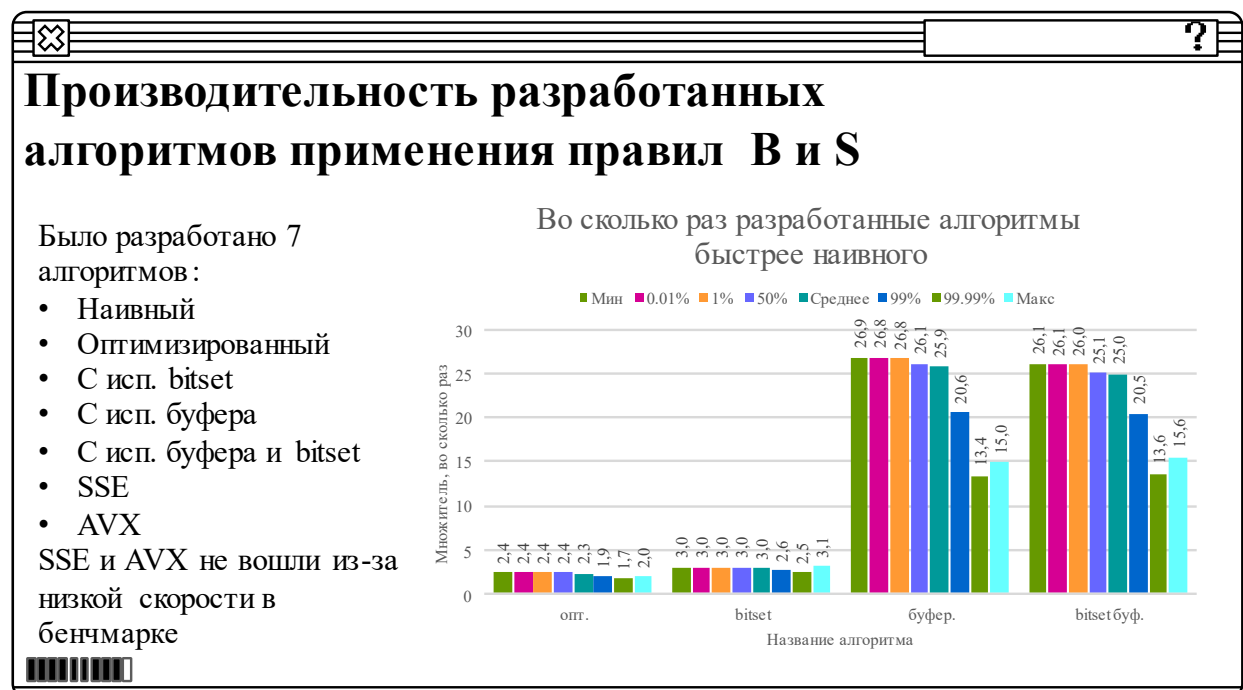


Рисунок Г.9 – Производительность разработанных алгоритмов применения правил В и S



Рисунок Г.10 – Производительность общих алгоритмов

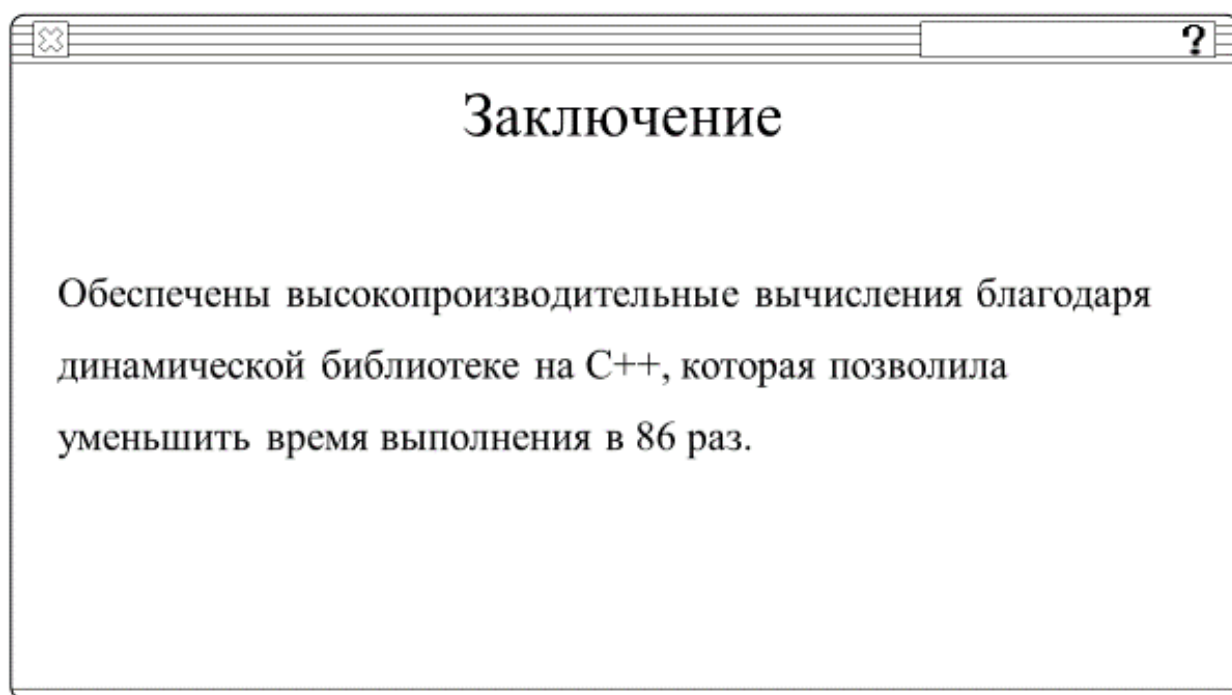


Рисунок Г.11 – Заключение